לימוד תכנות בפייתון

מבוסס על תוכן שפותח על ידי המרכז לחינוך סייבר



python™

מבוא

לימוד תכנות בפייתון.

עריכה: נתנאל מרחום.

תוכן: המרכז לחינוך סייבר מתוך קורס פייתון באתר קמפוס.

גירסת פייתון: 3.7 התוכן מותאם גם לגרסאות מעל 3.

©התוכן מבוסס על קורס "self.py – הדרך שלך ללמוד פייתון" של מוקד הידע הלאומי, שהוקם ע"י המרכז לחינוך סייבר, קרן רש"י ומערך הסייבר הלאומי במשרד ראש הממשלה.

הקורס המלא זמין **ללא עלות** באתר קמפוס IL.

לעוד קורסים מצוינים בחינם של המרכז לחינוך סייבר.

אין לראות בספר שהוא מטעם המרכז לחינוך סייבר אלא מבוסס על תוכן שפותח על ידי המרכז לחינוך סייבר

להערות: mmm040846@gmail.com.

אשמח שתענו על המשוב הקצר - משוב ספר פייתון.

תוכן עניינים

מאפיינים של שפות נמוכות ושל שפות עיליות 10

יתרונות וחסרונות של שפה עילית 10

מאפיינים של שפות נמוכות ושל שפות עיליות 10

יתרונות וחסרונות של שפה עילית 10

האינטרפרטר 11

פעולות חשבון בסיסיות 11

הפקודה print 12

הערת שורה 12

שימוש בספריות 12

מהו משתנה 13

טיפוסי משתנים בסיסיים 13

פעולות בסיסיות עם משתנים 14

שינוי ערך של משתנה: 14

השמה: 14

תיעוד הפונקציה help: 14

טיפוס בולאני 15

ביטויים לוגים.....15

קבלת קלט מהמשתמש.....16

המשתנה האוטומטי "_" 16

החלפה בין משתנים.....16

מהן קונבנציות?.....17

מתן שמות למשתנים 17

קבועים 17

שמות משמעותיים 17

הגדרת מחרוזת.....18

דרכים נוספות ליצור מחרוזת 19

תווים מיוחדים בהדפסת מחרוזות.....20

סימון מחרוזת ארוכה.....20

גישה לתו במחרוזת.....21

גישה מהסוף לתו במחרוזת 21

חיתוך, מקטעים וקפיצות.....22

קפיצות 23

שינוי מחרוזת.....23

קונבנציות - רווחים 24

פונקציות.....24

מתודות 25

הפונקציה dir 25

עבודה עם עורך טקסט 26

בלוקים - תחביר וקונבנציות 26

הזחה 26

הפקודה pass 26

בלוק תנאי 27

התנאי if 27

ביטויים לוגיים בתנאים 27

התנאי elif 28

התנאי else 28

כתיבת פונקציות וקריאה לפונקציות 29

פונקציה היא אובייקט 30

מה בין פרמטר לארגומנט? 30

טווח ההכרה עבור משתנים(scope) 31

משתנה לוקאלי 31

משתנה גלובלי 31

גישה למשתנה לוקאלי מחוץ לטווח ההכרה 31

גישה למשתנה גלובלי 32

פלט וקלט של פונקציה.....34

החזרת ערך אחד מפונקציה.....34

החזרת שני ערכים ומעלה.....34

ערכי ברירת מחדל לארגומנטים.....35

ארגומנטים דיפולטיים 35

ארגומנטים שאינם דיפולטיים.....36

תיעוד פונקציה 36

צפייה בתיעוד הפונקציה.....38

הפונקציהmain 38

שם הפונקציהmain 39

משמעות התנאי בפונקציה הראשית.....40

הגדרה נכונה וכתיבה נכונה של 43

פונקציות.....43

קונבנציות בכתיבת פונקציות.....43

יצירת רשימה.....44

גישה לאיברים ברשימה.....45

יצירת רשימה באמצעותlist() 45

רשימה בתוך רשימה 46

שמירת איברי רשימה למשתנים.....47

דמיון בין מחרוזת לבין לרשימה.....47

גישה וחיתוך בעזרת מיקום.....48

שרשור.....49

בדיקת אורך.....49

רשימה היא - mutable.....50

הוספת איבר ברשימה.....50

שינוי.....50

מחיקת איבר מרשימה.....51

מחיקת איברים.....51

בדיקה האם איבר קיים ברשימה.....51

פעולות שימושיות נוספות עם רשימות.....52

פונקציה sorted עם מפתח:(key).....54

שימוש בלולאתwhile.....55

כיצד כותבים לולאת while בפייתון.....55

לולאת while אינסופית.....56

בקרת זרימה בלולאות.....56

הפקודה break.....57

הפקודהcontinue.....57

שימוש בלולאות for.....58

בפייתון forכיצד כותבים לולאת 59

רשימה בתוך רשימה 60

לולאה שרצה מספר מסוים של פעמים 61

מבנה כללי של הפונקציהrange 62

מהו טאפל? 64

דמיון ושוני בין טפאל לבין רשימה 65

טאפל הואimmutable 65

בחירת מבנה נתונים 66

השמה באמצעות טאפל 67

החזרת ערכים מפונקציה 67

יצירת תבנית למחרוזת 68

מהו מילון? 71

יצירת מילון? 72

גישה לערך באמצעות מפתח 73

שינוי מילון - מילון הוא mutable 74

הוספת איבר למילון 74

עדכון איבר במילון 74

טיפוסי מפתחות וערכים 75

מחיקה של איבר ממילון 75

הצגת מפתחות או ערכים 76

הצגת מפתחות וגם ערכים 77

הצגה של מילון 77

שימוש בקבצים עבור קלט ופלט 78

פתיחת קובץ לקריאה: 78

התחילית: 79

סגירת קובץ 80

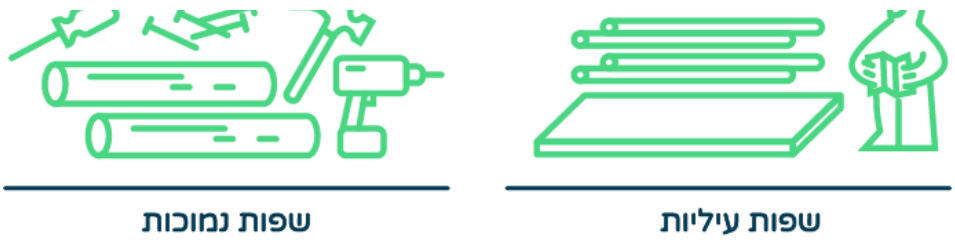
קריאת תוכן של קובץ 80

פתיחת קובץ לכתיבה 80

כתיבה לקובץ 80

בסיס השפה

לפני שנצלול להשוואה בין השפות, דמיינו שיש לכם צורך בשולחן חדש.
באפשרותכם **לבנות** אותו מן היסוד מלוחות עץ ומסמרים, ולקבוע בדיוק איך יראה כל פרט.
אולם, יהיה יותר נוח והרבה יותר אלגנטי לבחור חלקים (כגון סוגי רגליים, צורת פלטה) מתוך חלקי שולחן מוכנים ו**להרכיב** את השולחן מהרכיבים שבחרתם. כמובן, זה גם ימשך פחות זמן.



בהתאמה לדימוי של השולחן, נסווג את שפות התכנות לשתי קטגוריות :**שפה נמוכה** מקבילה לאפשרות של בניית שולחן מן היסוד. לעומתה ,**שפה עילית** מקבילה לאפשרות של הרכבת שולחן מחלקים מוכנים.
פייתון היא דוגמה לשפה עילית - שפה נוחה ואלגנטית שמאפשרת לכתוב ואף "להרכיב" תוכנית בעזרת רכיבים שאנו בוחרים, בקלות ובמהירות, לעיתים במחיר של ויתור על שליטה מוחלטת.

מאפיינים של שפות נמוכות ושל שפות עיליות

שפות נמוכות (Low level)הן שפות תכנות שקרובות יותר לשפת מכונה, שהיא השפה שהמעבד מבין, ולכן קצת פחות אינטואיטיביות עבורינו .

שפות נמוכות הן בדרך כלל **שפות מקומפלות** (compiled).בשפות אלה, בכדי לייצר קובץ בשפת מכונה שהמחשב ומערכת ההפעלה יכולים להריץ (לדוגמה קובץ הרצה עם סיומת EXE) הקוד עובר שלב של קומפילציה וקישור (linking).

דוגמאות לשפות נמוכות הן שפת C ושפת C++.

שפות עיליות (High level) הן שפות תכנות שקרובות יותר לשפת הדיבור שלנו ולכן הן קריאות וברורות יותר. שפות עיליות הן בדרך כלל **שפות מפורשות** (interpreted). בשפות אלה, תוכנה שנקראת אינטרפרטר ,(interpreter) מפרש (רצה, קוראת את הקוד, מתרגמת את הקוד ה"עילי" לקוד מכונה ומבצעת אותו. כלומר, בכדי שקובץ ירוץ חייבת להימצא על המחשב תוכנה מפרשת (אינטרפרטר).

יתרונות וחסרונות של שפה עילית

שפת **פייתון** היא דוגמה לשפה עילית, ובדומה לכך גם ג'אווה ו-c#.

לשפות עיליות **יתרונות** רבים על פני שפות נמוכות: אפשר ללמוד אותן ביתר קלות, הקוד פשוט וקריא יותר, זמן הפיתוח קצר יותר ולרוב הן כוללות יותר אפשרויות שמובנות בשפה עצמה (ללא צורך בייבוא חיצוני של קוד).

אם כך, למה שלא נתכנת תמיד בשפות עיליות?

התשובה היא שליתרונות הללו נלווים כמובן גם **חסרונות** ,לדוגמה:
משך הריצה של תוכניות הכתובות בשפה עילית ארוך יותר ביחס למשך הריצה של תוכניות הכתובות בשפה נמוכה. כמו כן, הן צורכות הרבה זיכרון. כיוון שאנו, המתכנתים, רחוקים יותר משפת המכונה שמדברת ישירות עם המעבד, יש לנו פחות שליטה על התוכנה ולעיתים אנו מוגבלים בפעולות שביכולתנו לבצע.

למעשה, כשאנו בוחרים שפת תכנות, אנחנו בוחרים היכן להתמקם על הטווח שבקצהו האחד פשטות וקריאות ובקצהו השני כוח ויעילות של התוכנה. פייתון, אולי ניחשתם, קרובה יותר לקצה הפשטות והקריאות, עובדה שלעיתים באה על חשבון השליטה והיעילות המרבית.



פרק 1 – היכרות עם השפה

האינטרפרטר

רגע לפני שנוכל להתחיל לכתוב קוד בפייתון, נכיר את סביבת העבודה שלנו. איפה נכתוב את הקוד? כיצד נריץ אותו? איך ייראה הפלט?

אחד היתרונות הבולטים של פייתון כשפה עילית הוא נוחות העבודה עם האינטרפרטר האינטרפרטר הוא התוכנה שמפרשת את הקוד שלנו וממירה אותו לקוד מכונה על ידי התוכנה (python.exe) בקיצור זהו חלון קטן בו רצה שפת פייתון.

האינטרפרטר מיוחד בכך שבכתיבת הקוד בו **אינטראקטיבית**: באינטרפרטר נכתוב בכל פעם שורת קוד אחת נלחץ אנטר ונקבל את הפלט בשורה הבאה.

```
>>> "Welcome to Python!"
'Welcome to Python!'
```

פעולות חשבון בסיסיות

פעולה	אופרטור	דוגמת הרצה
חילוק שתוצאתו היא שבר עשרוני.	/	`>>> 5 / 2` `2.5`
חילוק שתוצאתו היא שבר עשרוני ללא שארית.	//	`>>> 5 // 2` `2`
מודולו , תוצאתו היא שארית מפעולת החילוק. כלומר הוא מחלק את המספרים ואת כופל את השארית במספר שחילקנו	%	`>>> 5 % 2` `1`
חזקה.	**	`>>> 5 ** 2` `25`

פרק 1 - היכרות עם השפה

הפקודה print

פקודה מבצעת הדפסה של הערך שנתון לה בסוגריים דוגמא:

```
>>> print(“hello”)

Hello
```

הערת שורה

בכתיבת קוד נוכל שלב הערה כדי שאם חסר משהו בקוד נוכל להשלים אותו אחר , כך וזה טוב לקריאת קוד של אחרים.

נכתוב # ולאחריה טקסט חופשי , ה# מסמן לפייתון להעלם מהטקסט הסולמית עד סוף השורה.

```
>>> print("Wonder Woman") # Print my favorite superhero
Wonder Woman
```

שימוש בספריות

ניתן לייבא ספריות ואפשר גם להוריד מהאינטרנט ולייבא באמצעות import + שם הספרייה.

בכדי לקרוא לפעולה כלשהי, נציין את שם הספרייה + נקודה, ולאחר מכן את שם הפעולה.
בסוגריים נעביר את הערך הדרוש לפעולה.

```
>>> import math

>>> Math.cos(3.141592653589793)

-1.0
```

פרק 2 - משתנים בפייתון חלק א

מהו משתנה

בכל תוכנית שאנו כותבים נרצה להשתמש **במידע**: שמות, מספרים, תוצאות חישובים, סיסמאות ועוד... מידע זה עשוי להיות מידע שחישבנו או מידע שנקלט מהמשתמש. את המידע אנו מעוניינים לשמור, להציג, להעבירו לתוכנית אחרת, לבצע באמצעותו חישובים וכן הלאה.
משתנים מאפשרים לנו לאחסן מידע באופן זמני בזיכרון ולגשת אליו, ולכן הם דבר בסיסי וחשוב מאוד בכל שפת תכנות.

```
>>> x = 5
>>> print(x)
5
```

בכדי להבין טוב יותר את שפת פייתון וכיצד לתכנת בה, נחדד את ההבנה על האופן שבו פייתון מייצגת משתנים בזיכרון המחשב.

המשתנים בפייתון הם למעשה **מצביעים**. כל משתנה מצביע על **אובייקט**, כלומר על אזור שמוקצה **בזיכרון המחשב** ומכיל ערך כלשהו.

אם נבחן את הדוגמה הקודמת, נסיק ש x מצביע **לאובייקט** בעל הערך 5.

בפייתון, לכל אובייקט מוגדר **טיפוס** (type) אליו הוא שייך. האובייקט בעל הערך 5 הוא מטיפוס (int).







טיפוסי משתנים בסיסיים

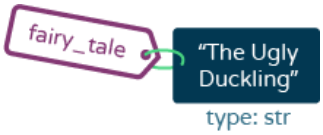
הטיפוסים הבסיסיים:

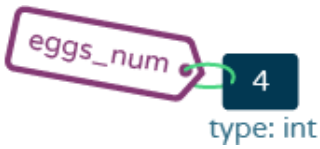
הטיפוסים הבסיסיים של משתנים בפייתון : string , int, float

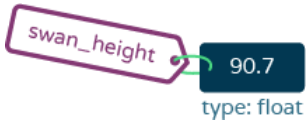
הטיפוס מחרוזת **string -** מכיל טקסט או כל דבר עם גרשיים(" ")

טיפוס **int -** היינו טיפוס של מספר בלי שארית (60.2)

טיפוס **float -** היינו טיפוס של מספר רק עם שארית (60.2)







פרק 2 – משתנים בפייתון חלק א

פעולות בסיסיות עם משתנים

שינוי ערך של משתנה:

בכדי **לשנות** ערך של משתנה, נסמן לפייתון שעליו להצביע על אובייקט אחר.

דוגמה לשינוי ערך של משתנה:

```
>>> swan_height = 90.7
>>> swan_height = "90.7cm"
```

שימו לב, כעת המשתנה swan_height מצביע על אובייקט מטיפוס מחרוזת (string) המכיל את הערך "90.7cm" חשוב להבין שלא מדובר בשינוי של המשתנה swan_height אלא בהצבעה שלו על ערך **שונה**, במקרה זה גם מטיפוס אחר.

השמה:

מה נעשה אם נרצה להגדיר משתנה נוסף ducklings_num, שמייצג את מספר הברווזונים? מספר הברווזונים שווה למספר הביצים ולכן נוכל לבצע **השמה** של המשתנה eggs_num למשתנה ducklings_num

```
eggs_num = ducklings_num
```

תיעוד הפונקציה help:

נוכל לבקש תיעוד (כלומר הסבר, פירוט) על פקודה מסוימת בעזרת הפקודה **help()**.

```
>>> help()
```

נוכל גם לבקש עזרה עבור פקודה, ספרייה או מילות מפתח בפייתון אם תוסיפו את המבוקש לפקודת **help** בסוגריים ומוקף במרכאות.

```
>>> help("in")
```

פרק 2 - משתנים בפייתון חלק ב

טיפוס בולאני

טיפוס חשוב ושימושי מאוד בפייתון הוא טיפוס בוליאני משתנה מטיפוס בוליאני הוא משתנה שמקבל את הערכים אמת או שקר בלבד.

ביטויים לוגים

משתנים בוליאניים משמשים הרבה לשמירת תוצאות של **ביטויים בוליאניים**, שנקראים גם **ביטויים לוגיים**. ביטוי לוגי הוא ביטוי שהתוצאה שלו היא חד משמעית - אמת או שקר.

שימו לב, פייתון תתייחס **לכל מספר** שאינו המספר 0 בתור הערך הבוליאני **True**, ואילו המספר 0 תמיד מייצג את הערך הבוליאני **False**.

אופרטור	תיאור	דוגמה לביטוי לוגי	תוצאת הביטוי לדוגמה
==	שווה	a == 2	True
!=	אינו שווה	b != 5	False
>	גדול מ-	a > 4	False
<	קטן מ-	a < 4	True
and	True אם תוצאת שני הביטויים היא True	a > 3 and b > 4	False
or	True אם תוצאת אחד מהביטויים (או שניהם) היא True	a > 3 or b > 4	True
not	הופך מ– True- ל- False ולהיפך	not(a > 4)	True
in	בדיקת הכלה במחרוזות (ובהמשך בטיפוסים נוספים)	"duck" in "duckling" "s" in "duckling"	True False

פרק 2 - משתנים בפייתון חלק ב

קבלת קלט מהמשתמש

אם נרצה לקבל קלט מהמשתמש כגון: איך קוראים לך? מהו מקום מגוריך?

נשתמש בפקודה input() שמאפשרת לנו לקבלת קלט מהמשתמש בהרצת פקודה זו ,התוכנית תחכה עד שהמשתמש ילחץ **Enter** לרוב נגדיר משתנה שיצביע על האובייקט שקלטנו מהמשתמש .

פייתון מתייחסת לקלט כאל **מחרוזת**

```
>>> name = input("Please enter your name:")
Please enter your name: yosi!
>>> print("Hello", name)
Hello yosi!
```

המשתנה האוטומטי "_"

בעת כתיבת קוד באינטרפרטר, תוכלו להיעזר במשתנה שנקרא "_" (קו תחתי). במשתנה זה נשמרת באופן אוטומטי התוצאה של הפעולה האחרונה שהתבצעה.

```
>>> 6
>>> _
6
>>> _ * 2
12
```

החלפה בין משתנים

נניח שהגדרנו שני משתנים ואנחנו רוצים להחליף בין הערכים שלהם.

```
>>> a = 1
>>> b = 2
>>> a, b = b, a
```

פרק 2 - משתנים בפייתון חלק ב

מהן קונבנציות?

קונבנציות כתיבה הן סט מוסכמות שנוגע לעיצוב ולנראות של הקוד, עליו מקפידים בכתיבת קוד. חשוב להדגיש, קונבנציות אינן נאכפות ברמת השפה, והתוכנית שלנו תרוץ גם בלעדיהן. למרות זאת, הן חשובות מאוד כיוון שהן מאפשרות גם לנו וגם לאחרים להבין ולהתמצא בקוד שלנו, או של מישהו אחר, בקלות.

מתן שמות למשתנים

נכיר כמה קונבציות ל**כתיבת שמות משתנים** בפייתון:

- שמות של משתנים נקפיד לכתוב ב**אותיות קטנות** .קיים שימוש גם באותיות גדולות, נכיר אותו בקרוב.
- אם נרצה לתת שם בעל שתי מילים או יותר - נפריד בין המילים בעזרת **קו תחתון**.

דוגמה לשם משתנה:

```
>>> current_book_sales = 5000
```

קבועים

מתי כן נשתמש באותיות גדולות?
לעיתים נרצה להגדיר ערך שאינו עתיד להשתנות במהלך ריצת התוכנית, משתנה מסוג זה נקרא **קבוע**.

- שמות קבועים נרשום ב**אותיות גדולות** בלבד, עם מילים מופרדות בקווים תחתונים.

שמות משמעותיים

בכדי שהתוכנית שלנו תהיה קריאה ובהירה, נרצה ששמות המשתנים ישקפו את המהות או או number המשמעות של הערך שהמשתנה מחזיק. הכוונה לשמות שאינם סתמיים כמו אפילו אם זה יצא מעט ארוך ,name.

- נבחר למשתנים **שמות משמעותיים** וברורים.

פרק 3 - מחרוזת

הגדרת מחרוזת

להזכירכם, מחרוזת הנה טיפוס נתונים מובנה בשפה שנקרא string. בפייתון כל רצף של תו אחד או יותר הנו מחרוזת, וכדי להגדירו עלינו להקיף את הטקסט שאנו רושמים ב - **מרכאות כפולות** (" ") או ב - **מרכאות יחידות** (' ').
המרכאות מסמנות לפייתון שמדובר באובייקט מטיפוס מחרוזת ולא בשם של פקודה או משתנה כלשהו.

כאשר מעוניינים שמחרוזת כלשהי תכלול גרש או מרכאות, אפשר להיעזר בפתרונות הבאים:

- אם המחרוזת מכילה מרכאות כפולות, נתחום אותה במרכאות יחידות .
  דוגמה למחרוזת שמכילה מרכאות כפולות :

```
>>> '"All in!", she said happily.'
'"All in!", sh e said happily.'
```

- אם המחרוזת מכילה גרש, נתחום אותה במרכאות כפולות .
  דוגמה למחרוזת שמכילה גרש :

```
>>> "Don't do it. You will lose the poker game."
"Don't do it. You will lose the poker game."
```

- אם המחרוזת מכילה גם מרכאות וגם גרש, נוכל להיעזר בתו מיוחד שמסמן לפייתון להתייחס למרכאות/לגרש בתוך המחרוזת כאל תווי טקסט רגילים .
  התו המיוחד נקרא **לוכסן הפוך** הלוכסן ההפוך נראה כך \ ונכתוב אותו **לפני** כל מרכאות או גרש שבתוך המחרוזת. שימושים נוספים של הלוכסן ההפוך נפרט בהמשך .
  דוגמאות לשימוש בלוכסן הפוך בהדפסת מחרוזות :

```
>>> print("\"Play with me\", she said.\"It will be SO
hilarious when you will lose!\"")
"Play with me", she said." It will be SO hilarious when
you will lose!"
```

פרק 3 - מחרוזת

דרכים נוספות ליצור מחרוזת

אפשר ליצור מחרוזות על ידי **שרשור** מחרוזות קיימות, באמצעות סימן החיבור (הסימן +).

דוגמה לשרשור מחרוזות:

```
>>> str1 = "We "
>>> str2 = "belong "
>>> str3 = "together"
>>> print(str1 + str2 + str3)
We belong together
```

אפשר **להכפיל** מחרוזת באמצעות סימן הכפל (הסימן *) ובכך ליצור מחרוזת חדשה .
דוגמה להכפלת מחרוזת:

```
>>> duplicate_me = ":-) "
>>> print(duplicate_me * 8)
:-) :-) :-) :-) :-) :-) :-) :-)
```

אפשר **להסב** משתנה שאינו מטיפוס מחרוזת בעזרת הפקודה , ()str כלומר להפוך אותו לטיפוס מחרוזת .
דוגמה להסבת משתנה למחרוזת:

```
>>> x = 3
>>> type(x)
<class 'int'>
>>> x + x
6
>>> x = str(x)
>>> x
'3'
>>> type(x)
<class 'str'>
>>> x + x
'33'
```

פרק 3 - מחרוזת

תווים מיוחדים בהדפסת מחרוזות

נכיר שני תווים מיוחדים שמשמשים בהדפסת מחרוזות. נשתמש בהם כאשר נרצה להדפיס מחרוזת ארוכה כך שתראה יותר מסודרת ולאו דווקא בשורה אחת רצופה:

- \n - שורה חדשה
- \t - הזחה (טאב)

כדי לרדת שורה (כלומר להתחיל שורה חדשה) נכתוב \n במקום הרצוי במחרוזת.

```
>>> print("Hey\nStarting a new line!")
Hey
Starting a new line!
```

כדי ליצור הזחה, נכתוב \t באופן דומה:

```
>>> print("Creating four spaces…\tDone!")
Creating four spaces…    Done!
```

סימון מחרוזת ארוכה

קיימים מקרים בהם נרצה לכתוב מחרוזת המתפרשת על פני כמה שורות. פייתון מאפשרת את זה בקלות על ידי **מרכאות משולשות**: הוספת רצף של ''' או של """ בתחילת המחרוזת ובסיומה.

```
>>> print("""How to win a chess game: A beginner's
manual.
    Step 1. Set the chessboard.
    Step 2. Find a bored friend.
    Step 3: Win the game.""")
How to win a chess game: A beginner's manual.
Step 1. Set the chessboard.
Step 2. Find a bored friend.
Step 3. Win the game.
```

פרק 3 - מחרוזת

גישה לתו במחרוזת

פייתון היא שפה נהדרת לעבודה עם מחרוזות ולכן כדאי לנו ללמוד כיצד לגשת לאיברים, מקטעים וחלקים שונים במחרוזת.

הגישה לתווים במחרוזת היא על פי המיקום שלהם, ולא על פי הערך שלהם. כלומר, כדי לגשת לתו "m" במחרוזת "Take me please" נכתוב לפייתון פקודה שמתייחסת לתו **במיקום** 5 במחרוזת, ולא מפורשות לערך m.

אם כך, בכדי לגשת לתווים במחרוזת נרצה לדעת היכן הם נמצאים בתוכה. כל תו במחרוזת נמצא במיקום(אינדקס , index)מסוים, כאשר המיקום הראשון הוא **אפס**(0)



הגישה לתו על פי מיקומו נעשית על ידי שימוש בסוגריים מרובעים [ ].

```
>>> card_str = "Joker"
>>> card_str[0]
'J'
>>> card_str[2]
'k'
>>> card_str[4]
```

גישה מהסוף לתו במחרוזת

אפשר לגשת לתו במחרוזת גם מהכיוון ההפוך, כלומר לספור מסוף המחרוזת באמצעות מיקומים שליליים.

דוגמה למיספור מיקומים במחרוזת מהסוף להתחלה וגישה לתווים:




```
>>> card_str = "Joker"
>>> card_str[-1]
'r'
>>> card_str[-3]
'k'
>>> card_str[-5]
'J'
```

פרק 3 - מחרוזת

חיתוך, מקטעים וקפיצות

לאחר שניגשנו לתווים בודדים במחרוזת נראה כיצד לחתוך מחרוזות ולגשת לחלקים או מקטעים שונים של המחרוזת. פעולת החיתוך נקראת **slicing** , ובעברית **חיתוך.**

החיתוך נעשה על ידי ציון המיקום (אינדקס) של תחילת החיתוך (START) והמיקום של סיום החיתוך (STOP) , וביניהם נקודתיים.

```
>>> # Strings slicing:

string[START:STOP]
```

שימו לב שהמספר שנכתוב כמיקום STOP יציין את המיקום שבו יסתיים החיתוך , אבל הוא עצמו **אינו** נכלל במקטע הנוצר. כך אורך המקטע יהיה תמיד ההפרש בין האינדקסים.

```
>>> hobby = "Gaming"
>>> hobby[0:2]
'Ga'
```

בדוגמה זו ניגשנו למיקומים 0 עד 2 (לא כולל), ואורך המחרוזת שנוצרה הוא 2

```
>>> hobby = "Gaming"

>>> hobby[2:5]
'min'
```

בחיתוך מקטעים אפשר להשתמש גם במיקומים שליליים.

```
>>> hobby = "Gaming"
>>> hobby[1:-1]
'amin'
```

בדוגמה זו ניגשנו למקטע ממיקום אחת עד מיקום מינוס אחת (האחרון), לא כולל.

פרק 3 - מחרוזת

קפיצות

אם לא התרשמתם מיכולות פייתון עד כה, נוסיף מרכיב מעניין נוסף!

קיימת אפשרות לחיתוך מחרוזות שכוללת קפיצה, או step, במרווחים שונים מאחד.
תבנית הכתיבה היא:

```
>>> # Slicing with step: string[START:STOP:STEP]
```

הערך האחרון שהתווסף (STEP) מציין את **מרווח הקפיצה**.

דוגמה לשימוש בחיתוך עם קפיצה: בדוגמה הבאה ניגש לכל התווים במיקומים הזוגיים - ממיקום אפס עד למיקום חמש (לא כולל), בקפיצות של 2.




```
>>> hobby = "Gaming"
>>> hobby[0:5:2]
'Gmn'
```

שימו לב שהשימוש בקפיצה מאפשר לנו לגשת בקלות לתווים זוגיים או אי זוגיים במחרוזת, מבלי להשתמש בלולאות!

שינוי מחרוזת

בואו ננסה לשנות תו בודד במחרוזת:

```
>>> my_string = "Change me!"
>>> my_string[0] = "S"
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: 'str' object does not support item assignment
```

קיבלנו הודעת שגיאה!

הסיבה לכך היא שאובייקטים מטיפוס מחרוזת הם בלתי ניתנים לשינוי (immutable). בכדי לשנות את המשתנה, במקרה זה my_string, נצטרך להגדיר אובייקט חדש מטיפוס מחרוזת עם השינוי הרצוי ולהעביר את my_string להצביע אליו. כלומר, נגדיר מחדש את my_string באופן הבא:

אם כך, שינוי ערכו של משתנה מטיפוס מחרוזת הוא למעשה הסרת ההתייחסות (ההצבעה) אל המחרוזת המקורית והפנייתה למחרוזת חדשה.

פרק 3 - מחרוזת

קונבנציות - רווחים

כאשר אנחנו מעבירים לפונקציה או למתודה **יותר מפרמטר אחד** בסוגריים, יש לכתוב רווח אחד אחרי כל פסיק שמפריד בין הערכים. אם מעבירים **פרמטר אחד** לא יהיו רווחים כלל.

שימו לב שכאשר כותבים קוד שכולל אופרטורים (כגון +, -, *, =, <,>) נדאג להכניס רווחים משני הצדדים שלהם. לדוגמה:

```
>>> x = 1              # Wrong: x=1
>>> x = x * 5 + 1      # Wrong: x=x*5+1
>>> x = a > 5          # Wrong: a>5
```

פונקציות

פונקציה היא כינוי לבלוק של קוד המבצע פעולה מסוימת. בכדי לבצע את הפעולה אפשר להשתמש בשם השמור של הפונקציה במקום לכתוב בכל פעם מחדש קוד המבצע זאת.

אופן השימוש בפונקציה - נכתוב את **שם הפונקציה** ולאחר מכן סוגריים. בתוך הסוגריים יופיע אובייקט, אחד או יותר, הדרושים לביצוע הפעולה. אובייקטים שונים נפריד בפסיקים. האובייקטים שדרושים לביצוע הפעולה נקראים **פרמטרים**.

```
function_name(parameter1, parameter2...)
```

מחשבת שורש ריבועי math המוגדרת בספרייה **sqrt** דוגמה לפונקציה - הפונקציה המתמטית (square root):

```
>>> import math
>>> math.sqrt(9)
3.0
>>> math.sqrt(25) + math.sqrt(14) + math.sqrt(9) +
math.sqrt(900) + math.sqrt(210) + math.sqrt(12)
59.69713574810113
```

ראיתם באיזו קלות חישבנו את הביטוי? פשוט על ידי קריאה חוזרת לאותה פונקציה. במקרה זה, היא פונקציה מובנית sqrt לשמחתנו הפונקציה math. מתוך הספרייה sqrt קראנו לפונקציה בשפה, כלומר היא מגיעה מוכנה לשימוש.

פרק 3 - מחרוזת

מתודות

מתודה היא פונקציה אשר מוגדרת עבור אובייקט מטיפוס מסוים, ומבצעת פעולות עליו או באמצעותו.

אופן השימוש במתודה - נכתוב מהו האובייקט עליו תפעל המתודה, נקודה, שם המתודה ולבסוף סוגריים. גם מתודות יכולות לקבל פרמטרים בסוגריים, ממש כמו פונקציות.

```
object.method_name(parameter1, parameter2...)
```

השימוש בנקודה מייצג קריאה למתודה עבור אובייקט, בשונה מקריאה לפונקציה מתוך ספרייה כפי שהכרנו.

מעניין לציין, גם שם האובייקט שכותבים(object)הוא פרמטר. כלומר, מאחורי הקלעים פייתון מעבירה למתודה את object בתור הפרמטר הראשון, ואחריו את הפרמטרים הנוספים שבסוגריים.

דוגמה למתודה שאינה מקבלת פרמטרים בסוגריים - המתודה upper מבצעת את פעולתה על אובייקט מטיפוס מחרוזת. כיוון שאין צורך בפרמטרים נוספים הסוגריים נשארים ריקים (אך נכתבים):

```
>>> "print me in UPPERCASE".upper()
'PRINT ME IN UPPERCASE'
```

בדוגמה האובייקט עליו מתבצעת הפעולה upper הוא המחרוזת "print me in UPPERCASE"

דוגמה למתודה אשר מקבלת פרמטר בסוגריים - המתודה **zfill** מרפדת מחרוזת (טיפוס האובייקט עליו היא פועלת) באפסים משמאל למחרוזת, עד לרוחב כלשהו (width). הרוחב הרצוי הוא מספר שלם שמועבר כפרמטר:

```
>>> "1".zfill(5)
'00001'
```

```
>>> "234".zfill(5)
'00234'
```

הפונקציה dir

מקבלת כפרמטר כל טיפוס או אובייקט בפייתון ומחזירה כפלט רשימה של המתודות הזמינות עבורו.

```
>>>  dir (str)
```

פרק 4 - תנאים

עבודה עם עורך טקסט

עד כה, לרוב, כתבנו את הקוד שלנו באינטרפרטר. האינטרפרטר מיועד לכתיבת קוד קצר ובדיקת חלקים קטנים של קוד. אך אם נרצה, ובעיקר בתוכניות גדולות, אפשר לכתוב קוד גם **בעורך טקסט.** בעורך טקסט נוכל לכתוב קוד, לשמור אותו כקובץ בעל הסיומת py , ולהריץ אותו.

עורך טקסט, כשמו, מאפשר עריכה נוחה יותר של הקוד. בכדי לכתוב קובצי פייתון אפשר לעבוד עם כל עורך טקסט, אפילו כתבן(notepad , pycharm) בסיסי עם זאת, קיימים עורכי טקסט אשר מסייעים לנו בכתיבת הקוד על ידי סימון חלקי קוד בצבעים, השלמה אוטומטית, הקפדה על קונבנציות ואף עזרה באיתור בעיות בכתיבת הקוד. עורך טקסט כזה מקל ומייעל את העבודה.

בלוקים - תחביר וקונבנציות

כשכותבים תוכנית היא כוללת לרוב "**בלוקים**": פסקאות של מספר שורות קוד ששייכות לאותו רצף לוגי. לפני שנכיר את בלוק התנאי, נכיר את המבנה הכללי של הבלוק הפייתוני.

```
if (age < 18):
    print("see you next time")
    guests -= 1
```

מהם מאפייני הבלוק?

השורה הראשונה בבלוק היא **שורת הפקודה** ולאחריה יגיע רצף שורות קוד ששייך לבלוק.

שורת הפקודה תמיד מסתיימת ב**נקודתיים** (כמו פקודת התנאי, if,בדוגמה) ויתר שורות הבלוק נמצאות בהזחה לשורה הראשונה.

הזחה (אינדנטציה , indentation) היא הרחקה של שורה מהשוליים.

הזחה

ההזחה בפייתון יכולה להתבצע על ידי 4 רווחים או על ידי מקש TAB.

הפקודה pass

לעיתים בכתיבת תוכנית אנחנו פותחים בלוק אבל עדיין לא "ממלאים" אותו בקוד. במקרה כזה נשתמש בפקודה שמדלגת - pass.
פקודה זו שימושית בין השאר כשרוצים להריץ חלק מתוכנית כדי לבדוק אותה, אך לפני שכתבנו חלקים אחרים .
דוגמה לשימוש ב-pass:

```
if python_grade > 90:
    print("I'm so proud of you!")
else:
  pass # TODO: print(<Add a motivational statement>)
```

פרק 4 - תנאים

בלוק תנאי

ודאי נתקלתם בחייכם בצורך לקבל החלטות, כלומר לבחור איך להתנהג בהינתן תנאים מסוימים. לדוגמה: **אם** התחזית גשומה **אז** אקח מטרייה ,**אם** צפויה שמש אצטייד בשמשייה ובכל מזג אוויר **אחר** - אצא עם שכמייה.

מבנה כזה מכונה **תנאי** ובתכנות הוא ממומש באמצעות הפקודה **if** ("אם"). פקודת תנאי (if) מאפשרת שליטה על סדר ביצוע (וגם אי-ביצוע) פעולות בתוכנית. כלומר, היא משמשת ל**בקרת זרימה** במהלך תוכנית.

פקודת תנאי מאפשרת ביצוע פעולה או פעולות, אם (ורק אם) **מתקיים** תנאי כלשהו. אפשר גם להגדיר ביצוע פעולה אחרת אם (ורק אם) התנאי **לא מתקיים.**

התנאי if

את התנאי נתחיל ב**מילת ההתנייה** ,**if** מיד אחריה נכתוב **מהו התנאי** שבודקים (ביטוי לוגי) ובסוף השורה נכתוב **נקודתיים**. לאחר מכן יתחיל בלוק **מוזח**. הבלוק מתאר את מה שיקרה בהינתן קיום/אי קיום התנאי ועליו להכיל לפחות שורת קוד אחת.

```
if 3 < 4:
  print("Three is less than four")
Three is less than four
```

ביטויים לוגיים בתנאים

בפקודות תנאי אפשר לשלב ביטויים לוגיים, כפי שלמדנו ביחידה על משתנים .
דוגמה לשילוב אופרטורים לוגיים ואופרטורים של השוואה בפקודת תנאי:

```
if (months_passed == 18) and (location_moon ==
location_sun):
   print("Eclipse!")
else:
   print("Just a regular day…")
```

בדוגמה זו מספר החודשים צריך להיות שמונה עשר, ומיקום הירח צריך להיות זהה למיקום השמש כדי שיודפס eclipse (ליקוי חמה). אחרת, יודפס Just a regular day (רק יום רגיל)…

פרק 4 - תנאים

התנאי elif

המילה השמורה **elif** (קיצור של else if, אחרת אם) משמשת להגדרת תנאי ביניים (אחד או יותר). פייתון בודקת נכונות של תנאי כלשהו ואם התנאי לא מתקיים, כלומר הבדיקה הראשונה החזירה שקר, היא ממשיכה לבדיקה הבאה.

elif יכולה להופיע **רק** לאחר משפט if . בסוף המקרים השונים שנבדקים עם elif אפשר (אך לא הכרחי) לשלב משפט else. משפט זה יתבצע אם אף אחד מהמצבים הקודמים לא התקיים.

```
choice = int(input())
if choice == 1:
  print("You've chosen one")
elif choice == 2:
  print("You've chosen two")
elif choice == 3:
  print("You've chosen three")
else:
  print("You've chosen something else")
```

התנאי else

המילה השמורה **else** (פרושה אחר) משמשת להגדרת תנאי אחר , פייתון בודקת נכונות של תנאי כלשהו ואם התנאי לא מתקיים, כלומר הבדיקה הראשונה החזירה שקר, היא תבצע את התנאי האחר כלומר את ה else.

שימו לב, השימוש במילה elif מסייע לנו להימנע מ**כתיבה מקוננת** ,כלומר תנאי בתוך תנאי. כתיבה מקוננת נחשבת מסורבלת, כפי שאפשר להתרשם מהשוואת הדוגמה הבאה, בה משתמשים ב -if ובב- else בלבד, לדוגמה הקודמת (שכוללת גם elif):

```
choice = int(input())
if choice == 1:
   print("You've chosen one")
else:
  if choice == 2:
     print("You've chosen two")
  else:
    if choice == 3:
       print("You've chosen three")
    else:
       print("You've chosen something else")
```

פרק 5 - פונקציות

כתיבת פונקציות וקריאה לפונקציות

פונקציה בפייתון מוגדרת עם המילה השמורה **def** (מלשון define , להגדיר) לאחריה שם הפונקציה ואז סוגריים ונקודתיים .
גם פונקציות הן בלוקים לוגיים של קוד, ולכן לאחר הנקודתיים תופענה הפקודות של הפונקציה כאשר הן מוזחות ימינה.

```
def my_first_function():
    print("This is my first function in Python!")
```

הקריאה לפונקציה נעשית על ידי כתיבת שם הפונקציה ולאחריה סוגריים (ללא רווח).

```
my_first_function()
This is my first function in Python!
```

דוגמה להגדרת הפונקציה square, המקבלת פרמטר:

```
def square(number):
    print(number ** 2)
```

בהתאמה להגדרת הפונקציה לעיל, בכל קריאה אליה היא תצפה לקבל פרמטר .
הפרמטר בדוגמה זו הוא מספר אותו מעלים בריבוע. לכן, כל קריאה לפונקציה מחוייבת בהעברת ערך מספרי בסוגריים.

איך מעבירים פרמטרים? בקריאה לפונקציה כותבים את הפרמטר בתוך הסוגריים. אם לא נקפיד על כך נקבל שגיאה מסוג TypeError .

דוגמה לקריאה לפונקציה square, עם פרמטר:

```
def square(number):
    print(number ** 2)

square(3)
9
```

פרק 5 - פונקציות

פונקציה היא אובייקט

נמשיך עם הפונקציה square. מה יקרה אם נכתוב את שם הפונקציה **ללא סוגריים** ונלחץ אנטר? האם נקבל הודעת שגיאה?

```
def square(number):
    print(number ** 2)

square
```

לא קיבלנו הודעת שגיאה ונראה שלא קרה דבר. מדוע ?
שם פונקציה ללא סוגריים הוא בעצם **מצביע** על פונקציה שיושבת בזיכרון התוכנית. אם נדפיס את שם הפונקציה ללא סוגריים, נקבל את מיקומה בזיכרון.

זה הזמן לציין שפונקציות, גם הן **אובייקטים** בפייתון!

אנו יכולים להתייחס אל פונקציות כמו לכל אובייקט אחר ולבצע השמה שלהן למשתנה אחר!
נגדיר משתנה חדש בשם func , ונגדיר את ערכו של func להיות הפונקציה squar :

```
def square(number):
    print(number ** 2)

func = square

func(6)
36
```

הפלא ופלא, עתה המשתנה func מצביע על פונקציה square שהגדרנו במקום אחר בקוד!

מה בין פרמטר לארגומנט?

עד כה התייחסנו למונח **פרמטר** ,שמייצג אובייקט שהפונקציה מקבלת לצורך פעולתה. את הפרמטר, או הפרמטרים, מגדירים בעת כתיבת הפונקציה .
כאשר קוראים לפונקציה מתוכנית כלשהי, שולחים עם הקריאה את האובייקט המוגדר לפונקציה. אובייקט זה, שמועבר בפועל בעת קריאה לפונקציה, נקרא **ארגומנט**.

```
def square(number):
    print(number ** 2)

square(3)
```

בדוגמה זו האובייקט number הוא הפרמטר שמוגדר לפונקציה .square מחובתנו להעבירו בעת קריאה לפונקציה. המספר שערכו 3 הוא הארגומנט, כלומר האובייקט המסוים שאנו מעבירים בתוכנית.

פרק 5 - פונקציות

טווח ההכרה עבור משתנים(scope)

פעמים רבות כאשר עובדים עם פונקציות, נרצה להעביר להן משתנים מתוך התוכנית. בכדי להבין כיצד לעבוד נכון עם משתנים בתוכנית, ובמיוחד בהעברתם לפונקציות, נתעכב על נושא חשוב והוא טווח ההכרה של משתנים בתוכנית.

טווח ההכרה (באנגלית **scope**) הנו הטווח שבו המשתנה קיים בתוך התוכנית. טווח ההכרה עשוי להיות גלובלי (עבור כל התוכנית) או לוקאלי (עבור קטע קוד מסוים) .
בהתאמה, משתנה בתוכנית או בפונקציה עשוי להיות משני סוגים :**משתנה לוקאלי** או **משתנה גלובלי**.

משתנה לוקאלי

משתנה לוקאלי הוא משתנה שטווח ההכרה שלו תחום למקום בו הוא הוגדר בלבד. לדוגמה, אם נגדיר משתנה בתוך פונקציה הוא יהיה לוקאלי לפונקציה. כלומר הוא מתקיים בפונקציה בה הוגדר בלבד ולאחר שריצת הפונקציה תסתיים הוא לא יהיה זמין לשימוש בתוכנית.

משתנה גלובלי

משתנה גלובלי הוא משתנה בעל טווח הכרה רחב, של התוכנית כולה.

הביטו בדוגמה הבאה:

```
global_var = "I'm global"

def f():
  local_var = "I'm local"
```

המשתנה global_var הנו משתנה גלובלי, כלומר - הוא מוכר **בכל התוכנית** .
לעומתו, המשתנה local_var הוא משתנה לוקאלי, כלומר משתנה **שמוכר אך ורק בבלוק או בפונקציה** שהגדירה אותו - במקרה זה הוא בטווח ההכרה של הפונקציה f.

גישה למשתנה לוקאלי מחוץ לטווח ההכרה

אם נגדיר משתנה **בתוך** פונקציה הוא יהיה לוקאלי לפונקציה. כלומר, טווח ההכרה שלו הוא אך ורק הפונקציה בה הוגדר, והוא לא יהיה זמין לשימוש בתוכנית לאחר שריצת הפונקציה תסתיים.

אם נריץ מחוץ לפונקציה משתנה לוקאלי זה ייתן שגיאה למה? כי טווח ההכרה שלו רלוונטי עבור הפונקציה בלבד.

כעת בואו נראה מה קורה כאשר **בתוך** פונקציה ניגשים למשתנה גלובלי שהוגדר בתוכנית **מחוץ** לה. הביטו בקוד הבא, מה לדעתכם הפלט שיתקבל?

```
a = 0
def my_function():
    a = 3
    print(a)

my_function(  )
print(a)
```

כאשר קיימים שני משתנים בעלי אותו שם, אחד לוקאלי והשני גלובלי, למשתנה הלוקאלי יש עדיפות. כלומר, במהלך ריצת הפונקציה היא מחפשת את a במאגר המשתנים הלוקאליים לפני החיפוש במאגר הגלובלי. לכן קריאה לפונקציה my_function מדפיסה את ערכו של המשתנה a הלוקאלי, שבטווח ההכרה של הפונקציה. שורת ההדפסה האחרונה מדפיסה את המשתנה a הגלובלי, שנותר ללא שינוי.

גישה למשתנה גלובלי

לעיתים נרצה לקרוא ערך ממשתנה גלובלי. נוכל לעשות זאת בצורה הבאה:

```
a = 0
def my_function():
    print(a)

my_function()

0
```

כאמור, פייתון מחפשת את השם a במרחב השמות הלוקאלי, ואם אינו נמצא היא מחפשת את השם במרחב השמות הגלובלי. צורת כתיבה זו עשויה להוביל לטעויות רבות ולכן היא **<u>אינה</u>** מומלצת .

באופן כללי, רצוי שלא לגשת בתוך פונקציה למשתנים שהוגדרו מחוץ לה (שאינם קבועים), אלא להעביר אותם לפונקציה בצורה מסודרת, כארגומנטים. כך הפונקציה עצמאית ולא תלויה במשתנים אחרים (נסו לחשוב: מה יקרה אם אותו משתנה כלל לא הוגדר בעת הקריאה לפונקציה?)

על אף האמור, אם בכל זאת נרצה לגשת למשתנים גלובליים בתוך פונקציה, **כולל** שינוי ערכם, יש באפשרותנו לעשות זאת. בכדי ליידע את פייתון שהמשתנה a כבר קיים מחוץ לטווח ההכרה ואנו רוצים להתייחס אליו נשתמש בהצהרה **global** באופן הבא:

```
a = 0
def my_function():
    global a
    a = 3
    print(a)

my_function()
print(a)

3
3
```

זה עתה הכרנו שיטת עבודה לגישה ולשינוי של משתנים גלובליים בתוך פונקציות, והדגשנו שהשיטה אינה מומלצת. ננסה להבין מה הבעייתיות בכך בעזרת הדוגמה הבאה:

```
global global_value


def mystery():
    if global_value:
        print("What's going on here?")
    else:
        print("Hey, who printed me?")
```

כפי שאפשר לראות, הפעולה של הפונקציה mystery תלויה במשתנה global_value. זהו משתנה גלובלי שטווח ההכרה שלו מחוץ לפונקציה. לכן, בתוך הפונקציה לא ברור אילו קטעי קוד בתוכנית שמחוץ לה משפיעים עליה, ובהתאם לכך לא ברור כיצד הפונקציה תתנהג .
כאשר אנו כותבים סקריפטים קצרים הבעיה הזו אינה גדולה כיוון שאנחנו רואים, זוכרים ומבינים בקלות את כל השינויים והתלויות בין המשתנים בקוד. לעומת זאת, בתוכניות מורכבות, מסורבל לעקוב אחר כל המתרחש. בכתיבה כמו בדוגמה בלתי אפשרי להתייחס לקוד כחלקים לוגיים נפרדים, כיוון שקטעי קוד שונים בתוכנית עלולים להשפיע אחד על השני באופן בלתי צפוי.
כמתכנתים, נוצרת לנו בעיה גדולה בעריכת שינויים ובבדיקת כל חלק בפני עצמו .
לכן, נשתדל להימנע משימוש במשתנים גלובליים בתוך פונקציה ונעדיף להעביר אותם כארגומנטים. במקרים בהם אין לנו ברירה נצהיר על השימוש באופן מפורש בעזרת המילה global.

פרק 5 - פונקציות

פלט וקלט של פונקציה

פעמים רבות נרצה שפונקציה תבצע פעולה מוגדרת, ולאחר מכן תחזיר ערך שמייצג את תוצאת הפעולה. זהו למעשה ה**פלט** של הפונקציה .
החזרת ערך מפונקציה נעשית בעזרת הפקודה **return**. פקודה זו מאפשרת להחזיר כל טיפוס שנרצה.

החזרת ערך אחד מפונקציה

דוגמה להחזרת ערך מטיפוס - int הפונקציה add מקבלת שני מספרים ומחזירה את הסכום שלהם:

```
>>> def add(num1, num2):
        return num1 + num2
```

בעת הקריאה לפונקציה נוכל לכתוב:

```
>>> sum = add(15, 20)
>>> print(sum)
35
```

כך נוצר משתנה sum שמצביע על האובייקט בעל הערך 35. כמובן שהערך אשר התקבל ונשמר במשתנה sum **אינו מצביע על פונקציה**, אלא על **התוצאה** שחושבה.

החזרת שני ערכים ומעלה

אפשרות שימושית מאוד בפייתון היא היכולת להחזיר מפונקציה **יותר מערך אחד** .בכדי להחזיר כמה ערכים נכתוב את המילה return ולאחריה רשימת הערכים מופרדת בפסיקים(למעשה, הפונקציה תחזיר טיפוס נתונים מיוחד שנקרא Tuple , אותו נלמד בהמשך)

```
def sum_and_avg(a, b):
    sum_num = a + b
    avg_num = (a + b) / 2
     return sum_num, avg_num
```

בקריאה לפונקציה נקבל פלט של שני ערכים, ונשמור אותם למשתנים:

```
>>> x, y = sum_and_avg(3, 6)
>>> x
9
>>> y
4.5
```

פרק 5 - פונקציות

ערכי ברירת מחדל לארגומנטים

כפי שראינו, בעבודה עם פונקציות נרצה להעביר לפונקציה ארגומנטים. הערכים המועברים הם **קלט** של הפונקציה.

פייתון מאפשרת לנו לקבוע **ערך ברירת מחדל** עבור ארגומנט. כלומר, כל עוד לא ציינו מפורשות אחרת, הארגומנט שמועבר לפונקציה הוא בעל ערך ידוע שהגדרנו מראש. ארגומנט כזה קרוי לעיתים **ארגומנט דיפולטי** ,כיוון שהוא בעל ערך דיפולטי (=ארגומנט ברירת מחדל), או **ארגומנט אופציונלי** ,כיוון שהעברתו לפונקציה בצורה מפורשת אפשרית אך אינה הכרחית.

את ערך ברירת המחדל נכתוב בשורת ההגדרה של הפונקציה, לאחר הפרמטר הרלוונטי, עם הסימן =.
דוגמה להגדרת הפונקציה multiply עם ארגומנטי ברירת מחדל:

```
def multiply(base=4, multiplier=5):
    print(base * multiplier)
```

בהתאמה לשורת ההגדרה, מה יקרה אם נקרא לפונקציה multiply עם סוגריים ריקים?

```
>>> multiply()
20
```

בדוגמה זו, על אף שלפונקציה מוגדרים פרמטרים, קראנו לה עם סוגריים ריקים. כיוון **שציינו** בהגדרת הפונקציה את ערכי ברירת המחדל עבור כל פרמטר, כל עוד לא כתבנו אחרת, הארגומנטים שיועברו <u>בפועל</u> לפונקציה הם 4 ו-5 ויודפס הפלט 20.

כמובן שגם כאשר **מגדירים** ערכי ברירת מחדל אפשר לקרוא לפונקציה עם ערכים **ששונים** מהם:

```
>>> multiply(2, 3)
6
```

ארגומנטים דיפולטיים

אם נוסיף ערך ברירת מחדל רק לפרמטר הראשון ולשני לא נקבל הודעת שגיאה תחבירית שמפרטת שהוגדר פרמטר ללא ארגומנט ברירת מחדל אחרי פרמטר בעל ארגומנט ברירת מחדל

קריאה כזו עלולה לבלבל כיוון שפייתון לא יודעת האם הארגומנט 3 הוא עבור הפרמטר השני

לפיכך, פרמטרים עם ארגומנטי ברירת מחדל יבואו תמיד **אחרי** פרמטרים ללא ארגומנטי ברירת מחדל. כלומר, רק לפרמטרים **האחרונים** ברשימת הפרמטרים (הפרמטרים מימין) יכולים להיו ארגומנטי ברירת מחדל.

פרק 5 – פונקציות

בקריאה לפונקציה בפייתון נוכל גם לציין את **שמות הפרמטרים** באופן מפורש, ובכך איננו מתחייבים לסדר הפרמטרים שהוגדר בעת כתיבת הפונקציה. שיטה זו מקלה בקריאה לפונקציה מרובת פרמטרים והופכת את הקוד לקריא יותר.

```
def show_date(day ,month, year=2017):
    print("Today is ",day,"/",month,"/",year)

show_date(day=15,month=12)
```

ארגומנטים שאינם דיפולטיים

שימו לב, אם בחרתם בצורת כתיבה של שמות מפורשים, הקפידו להתייחס **לכל** הפרמטרים **שאין** להם ערכי ברירת מחדל .

דוגמה לקריאה לפונקציה show_date עם ציון חלקי של פרמטרים:

```
>>> show_date(day=15)
Traceback (most recent call last):
 File "<stdin>", line 1, in <module>
TypeError: show_date() missing 1 required positional
argument: 'month'
```

מתקבלת הודעת שגיאה כיוון שלא התייחסנו לפרמטר month **שאין לו** ערך ברירת מחדל.

תיעוד פונקציה

ה**תיעוד** של פונקציה מתאר מה מטרת הפונקציה ואיך להשתמש בה.
תיעוד הוא חלק בלתי נפרד מכתיבת קוד והוא שימושי מאוד הן בהתמצאות בתוכנית שלכם והן בקריאת קוד של אחרים .
לכן מומלץ שתתרגלו לכתוב תיעוד לפונקציות שכתבתם, ולהשתמש בתיעוד פונקציות שכתבו אחרים.

תיעוד לפונקציה נמצא מייד לאחר הגדרת הפונקציה ועליו לכלול **הסבר על מה שהפונקציה מבצעת** .התיעוד נכתב כשורה, אחת או יותר, בתוך הקוד. חלק זה נקרא **docstring** והוא ערוך בתוך מרכאות משולשות.

דוגמה לתיעוד פונקציה (docstring) המכיל 2 שורות:

```
def func():
  """This function does nothing for now, but I still
document it.
   Really, documentation is very important!"""
   <your code here>
```

אם אנו כותבים פונקציה **שמקבלת פרמטרים ו/או מחזירה ערכים**, נוסיף זאת לתיעוד לאחר השורה שמפרטת מה הפונקציה מבצעת.

```
def power(base, exponent):
   """Calculates a number raised to a power.
   :param base: base value
   :param exponent: exponent value
   :type base: int
   :type exponent: int
   :return: The result of raising base to the power
exponent
   :rtype: int
   """
   return base ** exponent
```

בשורה 2 מתועדת בקצרה **פעולת הפונקציה**.

- בהמשך התיעוד מתוארים הפרמטרים שהפונקציה מקבלת (אם יש כאלה), והערכים שהיא מחזירה. כל שורה נפתחת בנקודתיים.
- בשורה 3 ובשורה 4 מופיע פירוט **הפרמטרים שהפונקציה מקבלת**. כל פרמטר שהפונקציה מקבלת מתועד בשורה נפרדת על ידי המילה **param**, שם הפרמטר ואחריו נקודתיים, והסבר על הפרמטר.
- בשורה 5 ובשורה 6 מופיעים **הטיפוסים של הפרמטרים**. הטיפוסים של הפרמטרים מתועדים על ידי המילה **type**, שם הפרמטר (בהתאמה לשמות שתועדו קודם) ואחריו נקודתיים, וטיפוס הפרמטר. גם כאן, הטיפוס של כל פרמטר מתועד בשורה נפרדת.
- בשורה 7 מופיע פירוט על **הערכים שהפונקציה מחזירה**. ערך ההחזרה מתועד על ידי המילה **return**, נקודתיים, והסבר על ערך ההחזרה.
  שימו לב, אם אין בפונקציה שורת return שמחזירה ערך במפורש, נכתוב **None** בתיעוד ערך ההחזרה.
- בשורה 8 מופיע טיפוס הערך המוחזר. הטיפוס מתועד על ידי המילה **rtype** ונקודתיים וטיפוס הערך.

פרק 5 – פונקציות

צפייה בתיעוד הפונקציה

זוכרים את הפונקציה func ? כך נצפה בתיעוד שלה:

- דרך הפקודה (את שם הפונקציה)help

```
>>> help(func)
Help on function func in module __main__:

func()
  This function does nothing for now, but I still
document it.
  Really, documentation is very important!
```

- שם הפונקציה ומיד לאחר מכן __doc__.

הפונקציהmain

לפניכם תוכנית קצרה שמכילה שתי פונקציות:

```
def func1():
    print("func1")

def func2():
    print("func2")
```

כאשר אנו מריצים את פייתון על קובץ ההרצה הזה, **נדמה כי לא מתבצע דבר**. אך בפועל,
במהלך ריצת התוכנית פייתון **מגדירה כל אחת מהפונקציות** ורק לאחר מכן מסתיימת הריצה.
בכדי ש**הפונקציות עצמן ירוצו** אנו קוראים להן מתוך **פונקציה ראשית ששמה main.**

מעתה והלאה, הקפידו לכתוב תוכניות שכוללות את הפונקציה main() במבנה הבא:

```
def main():
    <your code here>

if __name__ == "__main__":
    main()
```

שימו לב למבנה הכתיבה שלעיל:

- בשורה 1 **מוגדרת הפונקציה הראשית** שנקראת main. בפונקציה זו, במקום השורה <your code here> , נהוג לאגד את כל מה שרוצים שיבוצע (קוד, או קריאות לפונקציות שונות).
- בשורה 5 **מתבצעת קריאה לפונקציה** main(), בכך מריצים את קטעי הקוד בפונקציה main() שהוגדרה זה מכבר.
- בשורה 4, טרום הקריאה לפונקציה main(), כתובה **פקודת תנאי** במבנה שטרם נתקלנו בו. למבנה זה יש כמה משמעויות שישרתו אתכם בהמשך הדרך, בעיקר בעבודה עם ספריות שתכתבו בעצמכם. אנו בטוחים שהדברים יתבהרו בהמשך דרככם כמתכנתי פייתון. הסבר בהמשך

חשוב להכיר את המבנה המקובל: צורת הכתיבה הזו נפוצה וסביר להניח שתיתקלו בה לא מעט כשתקראו קוד של מתכנתים אחרים .
בהרצת תוכנית הכתובה במבנה זה, הפונקציה הראשית main() היא **הפונקציה הראשונה שנקראת**.

```
def func1():
  print("func1")

def func2():
  print("func2")

def main():
  func1()
  func2()

if __name__ == "__main__":
  main()

func1
func2
```

שם הפונקציה main

שימו לב כי השם המקובל לפונקציה הראשית הוא , main() אך הוא בגדר **קונבנציה ולא תנאי** של השפה.

פרק 5 – פונקציות

משמעות התנאי בפונקציה הראשית

לפני שנדבר על מבנה הפונקציה ,main ניגע בקצרה בנושא חדש שיסייע לנו להבנת הנושא, והוא **יצירת קובץ ספרייה משלנו**.

עד כה למדנו לייבא ספריות קיימות, כדוגמת הספרייה math , ולהשתמש בפונקציות מתוכן. כעת נלמד כיצד ליצור **בעצמנו** קובץ ספרייה עם פונקציות שמימשנו ולהשתמש בהן.

לפנינו קובץ פייתון שנקרא my_math.py , ובתוכו פונקציה שנקראת add .
הפונקציה מקבלת כפרמטרים שני מספרים ומדפיסה את תוצאת החיבור שלהם:

```
my_math.py
def add(num1, num2):
    print("The result is:", num1 + num2)
```

בכדי לבדוק שהפונקציה אכן מבצעת את המצופה ממנה, נשלב קריאה לפונקציה ()add מתוך הפונקציה הראשית בקובץ. הקריאה לפונקציה מתבצעת בשורה 5 והקריאה לפונקציה הראשית ()main מתבצעת בשורה 7:

```
my_math.py
def add(num1, num2):
    print("The result is:", num1 + num2)

def main():
    add(3,1)

main()
The result is: 4
```

בהרצת הקובץ my_math.py מודפסת כמובן התוצאה 4.

למעשה, הקובץ my_math.py שכתבנו יכול לשמש גם בתור **ספרייה שאפשר לייבא עבור תוכנית אחרת .**כך, בקלות רבה, נוכל ליצור לעצמנו ספריות המכילות פונקציות על פי צרכינו.

עתה נלמד כיצד להשתמש בפונקציה המתמטית ()add שמימשנו בקובץ כחלק מתוכנית אחרת .

ניצור קובץ שנקרא another_program.py. במסגרת תוכנית זו נייבא באמצעות הפקודה import את הקובץ המכיל את הפונקציה ()add.

```
another_program.py
import my_math
```

הפקודה import רומזת לפייתון להתייחס לקובץ my_math.py כספרייה, ובכך מאפשרת להשתמש בפונקציות שמוגדרות בה.

הקריאה לפונקציות מתוך ספרייה מתבצעת כפי שכבר למדנו: על ידי ציון שם הספרייה, נקודה, ושם הפונקציה כולל סוגריים עם הפרמטרים שהיא מקבלת:

```
another_program.py
import my_math

my_math.add(5, 2)
The result is: 4
The result is: 7
```

שמתם לב לפלט? מדוע מודפסות שתי שורות פלט?

שורת ההדפסה הראשונה תואמת לקריאה add(3 ,1) שכתבנו כבדיקה בקובץ הספרייה my_math. לאחריה מופיעה שורת ההדפסה התואמת לקריאה add(5 ,2) שכתבנו בתוכניתנו החדשה.

בפועל, **יבוא הספרייה** my_math על ידי הפקודה import הוביל **להרצה של הקובץ כולו**, כולל הפונקציה הראשית שבו, שקוראת לפונקציה ()add.

המצב הרצוי הוא שהתוכנית שאנו כותבים תכיר את **הפונקציות שהגדרנו** בקובץ הספרייה **בלבד**, ותוכל לקרוא להן. זאת מבלי להריץ את התוכנית כולה הכתובה בקובץ הספרייה. המצב הזה מתאפשר על ידי שימוש במבנה התנאי עבור הפונקציה main:

```
def main():
    <your code here>

if __name__ == "__main__":
    main()
```

פקודת התנאי שמופיעה בשורה 4 מבחינה בין שני שימושים בקובץ:

1. **הרצה ישירה של הקובץ**. במקרה זה **מתבצעת קריאה לפונקציה ()main** שבמהלכה יורצו שורות הקוד והפונקציות הנוספות שמוגדרות בה.

2. **יבוא הקובץ כספרייה, באמצעות הפקודה import**. במקרה זה **הפונקציה ()main לא תרוץ** והקובץ ישמש כספרייה ובה פונקציות הזמינות לשימוש.

אם כן, נוסיף את מבנה התנאי לקובץ הספרייה my_math:

```
my_math.py
def add(num1, num2):
    print("The result is:", num1 + num2)

def main():
    add(3,1)

if __name__ == "__main__":
    main()
```

כעת נריץ שוב את הקובץ another_program.py (שכולל יבוא לקובץ הספרייה my_math וקריאה לפונקציה add(5 , 2) מתוכו) :

```
another_program.py
import my_math

my_math.add(5, 2)
The result is: 7
```

בזכות שורת התנאי שהוספנו הקובץ my_math.py מיובא כקובץ ספרייה, ואינו מורץ בשלמותו.
לכן הקריאה לפונקציה main() שבתוך קובץ הספרייה אינה מתבצעת ומודפסת התוצאה 7 בלבד.

פרק 5 – פונקציות

הגדרה נכונה וכתיבה נכונה של פונקציות

שרוצים לכתוב קוד רושמים בצורה ברורה ומפרטים הכל ולא רושמים בצורה מקוצרת.

```
def main():
  # City forcast generator program
  city_name = get_city_from_user()
  valid_city = is_city_valid(city_name)
  if not valid_city:
     print("Error! Wrong City!")
     return None
  temperature = get_current_weather(city_name)
  # Show forecast for the user
  show_weather(temperature, city_name)

if __name__ == "__main__":
  main()
```

קונבנציות בכתיבת פונקציות

- כתבו כל פונקציה עם **מטרה אחת ברורה** .כאשר המטרה ממוקדת נוכל להשתמש בפונקציה פעמים רבות ובתרחישים מגוונים ובכך אנו מונעים שכפול מיותר של קוד.

- **שם הפונקציה** צריך לשקף את המטרה וכך גם **תיעוד** שלה.

- השתדלו ש**הקלט והפלט** של הפונקציה יהיו ברורים והשימוש בהם אינטואיטיבי.

- בתוכנית עם יותר מפונקציה אחת, נפריד ביניהן באמצעות **שורה ריקה**

פרק 6 – רשימות

יצירת רשימה

רשימה מסמנים על ידי **סוגריים מרובעים [ ].** בכתיבת סוגריים מרובעים אפשר ליצור רשימה, אפילו ריקה .

```
>>> empty_list = []
>>> type(empty_list)
<class 'list'>
>>> print(empty_list)
[]
```

כדי ליצור רשימה המכילה אובייקטים, יש להפריד אותם באמצעות פסיקים. כל אובייקט מכונה **איבר** ברשימה .
דוגמה ליצירת רשימה המכילה את האיברים הראשונים בסדרת פיבונאצ'י (סדרה בה כל איבר הוא סכום שני האיברים הקודמים לו) :

```
>>> fibo_list = [1, 1, 2, 3, 5, 8, 13, 21]
>>> fibo_list
[1, 1, 2, 3, 5, 8, 13, 21]
```

הרשימה fibo_list מכילה שמונה מספרים, כלומר איברים מהטיפוס int , אולם איברים ברשימה יכולים להיות גם מטיפוסים שונים .

דוגמה לרשימה עם איברים מטיפוסים שונים:

```
>>> crazy_list = [2, "The Big Bang Theory!", True, 4.6]
>>> crazy_list
[2, 'The Big Bang Theory!', True, 4.6]
```

פרק 6 – רשימות

גישה לאיברים ברשימה

כל איבר ברשימה מוגדר על ידי **מיקום** (אינדקס). כלומר, קיימת חשיבות לסדר האיברים ברשימה .

בדומה למחרוזת, גם ברשימה המיקומים מתחילים מאפס, ואפשר לגשת לאיבר מסוים על ידי ציון המיקום שלו.

```
>>> crazy_list
[2, "The Big Bang Theory!", True, 4.6]
>>> crazy_list[0]
2
>>> crazy_list[1]
'The Big Bang Theory!'
>>> crazy_list[2]
True
>>> crazy_list[3]
4.6
```

יצירת רשימה באמצעות()list

למדנו כיצד ליצור רשימה באמצעות סוגריים מרובעים [ ]. דרך נוספת ליצירת רשימה חדשה היא בעזרת הפונקציה **list**.

```
>>> empty_list = list()
>>> empty_list
[]
```

באמצעות ()list אפשר גם ליצור רשימה ממחרוזת. זו אינה רשימה ריקה, אלא רשימה שכל איבר בה הוא תו במחרוזת המקורית .

```
>>> letters = "science"
>>> list_of_letters = list(letters)
>>> list_of_letters
['s', 'c', 'i', 'e', 'n', 'c', 'e']
```

פרק 6 – רשימות

רשימה בתוך רשימה

כפי שראינו, רשימה יכולה להכיל איברים מטיפוסים שונים. בהתאמה לכך, רשימה יכולה אף להכיל איברים מהטיפוס "רשימה."

```
>>> unique = [["a", "list"], ["of", "lists"]]
>>> unique
[['a', 'list'], ['of', 'lists']]
```

בואו נראה כיצד לגשת לאיברים של רשימה בתוך רשימה .

כאשר יש רשימה "חיצונית" המכילה רשימה "פנימית", ראשית יש לגשת למיקום של האיבר ה"חיצוני" ואז לגשת לאיבר ה"פנימי" הרצוי.

```
>>> unique
[['a', 'list'], ['of', 'lists']]
>>> unique[1]
['of', 'lists']
>>> unique[1][0]
'of'
>>> unique[0][1]
'list'
>>> unique [0][0]
'a'
```

ייתכן שתהיתם מתי שימושי להשתמש במבנה של רשימה בתוך רשימה. מבנה זה נוח לשימוש כאשר מארגנים יחד נתונים מסוגים שונים.

לדוגמה, לפניכם רשימה בה כל איבר מכיל רשימה שכוללת שם, שנת לידה ותחום עיסוק .
הביטו ברשימה ובאופן הגישה לנתונים ברשימה:

```
>>> influential_people = [["Mark Zuckerberg", 1984,
"Computer programmer"], ["Sheryl Sandberg", 1969,
"Technology executive"], ["Albert Einstein", 1879,
"Theoretical physicist"]]
>>> influential_people[0]
['Mark Zuckerberg', 1984, 'Computer programmer']
>>> influential_people[0][0]
'Mark Zuckerberg'
>>> influential_people[0][1]
1984
```

פרק 6 – רשימות

שמירת איברי רשימה למשתנים

לפניכם טריק שימושי בעבודה עם רשימות: אפשר לשמור בקלות איברי רשימה לתוך משתנים !

דוגמה להשמת איברי רשימה למשתנים:

```
>>> cool_trick = [1, 2, 3]
>>> a, b, c = cool_trick
>>> a
1
>>> c
3
```

שימו לב: עליכם להקפיד לייצר משתנים כמספר האיברים ברשימה אחרת מתקבלת הודעת שגיאה!

דמיון בין מחרוזת לבין לרשימה

כפי שראינו, קיים דמיון בין מחרוזת לבין רשימה - התווים במחרוזות כמו גם האיברים ברשימה ממוספרים ואפשר לגשת אליהם באמצעות המיקום שלהם .

דמיון זה נובע בשל היותן **רצפים** (sequences).

הכרנו פעולות של רצפים שאפשר לבצע על מחרוזת (כגון גישה, חיתוך, שרשור והכפלה), וגם רשימה תומכת בפעולות דומות.

נדגים את הדמיון בעזרת השוואה בין עבודה עם **המחרוזת** my_subject שתצביע לערך chemistry (המקצוע "כימיה"), לבין עבודה עם **הרשימה** chemical_elements שתכיל יסודות כימיים.

הגדרת המחרוזת my_subject והרשימה chemical_elements :

```
>>> my_subject = "chemistry"

>>> chemical_elements = ["hydrogen", "helium", "lithium", "beryllium"]
```

נשווה פקודות ופעולות בעבודה עם המחרוזת והרשימה שהגדרנו.

גישה וחיתוך בעזרת מיקום

מחרוזת	רשימה

`>>> my_subject`
`'chemistry'`
`>>> my_subject[3]`
`'m'`
	`>>> chemical_elements`
	`['hydrogen', 'helium',`
	`'lithium', 'beryllium']`
	`>>> chemical_elements[3]`
	`beryllium`

אפשר לגשת לתו/איבר גם באמצעות **מיקום שלילי**:

מחרוזת	רשימה

`>>> my_subject`
`'chemistry'`
`>>> my_subject[-1]`
`'y'`
	`>>> chemical_elements`
	`['hydrogen', 'helium',`
	`'lithium', 'beryllium']`
	`>>> chemical_elements[-1]`
	`beryllium`

באופן דומה בו מבצעים **חיתוך** (slicing) של מחרוזת, נוכל גם לגשת ל**מקטע**, כלומר לכמה איברים , ברשימה.

מחרוזת	רשימה

`>>> my_subject`
`'chemistry'`
`>>> my_subject[1:3]`
`'he'`
	`>>> chemical_elements`
	`['hydrogen', 'helium',`
	`'lithium', 'beryllium']`
	`>>> chemical_elements[1:3]`
	`['helium', 'lithium']`

פרק 6 – רשימות

שרשור והכפלה

כמו מחרוזות, אפשר **לשרשר** רשימות באמצעות סימן החיבור (הסימן +) .
דוגמה לשרשור מחרוזות ולשרשור רשימות:

```
>>> my_subject + " is awesome!"
'chemistry is awesome!'
>>> chemical_elements + chemical_elements
['hydrogen', 'helium', 'lithium', 'beryllium',
'hydrogen', 'helium', 'lithium', 'beryllium']
>>> chemical_elements + ["boron"]
['hydrogen', 'helium', 'lithium', 'beryllium', 'boron']
```

שימו לב שכאשר מעוניינים לשרשר אפילו ערך אחד הוא צריך להיכתב בצורת **איבר ברשימה** , כלומר תחום בסוגריים מרובעים [ ] .

הדבר מקביל לכך שלמחרוזת משרשרים תווים בתוך מרכאות (ולא מספר לדוגמה).

להזכירכם, למדנו שאפשר **להכפיל** מחרוזות, לדוגמה:

```
>>> my_subject * 4
'chemistrychemistrychemistrychemistry'
```

בהתאמה, אפשר להכפיל גם רשימות ליצירת רשימה חדשה ארוכה.

בדיקת אורך

לסיום, ישנן פעולות ופקודות רבות שהכרנו בעבודה עם מחרוזות שמשמשות גם בעבודה עם רשימות .

פונקציה שימושית היא הפונקציה **len** ,שמאפשרת לקבל **אורך** של רשימה (כלומר את מספר האיברים שהיא מכילה) .

```
>>> len(my_subject)
9
>>> len(chemical_elements)
4
```

פרק 6 – רשימות

רשימה היא - mutable

קיים הבדל מהותי בין מחרוזות לרשימות. בניגוד למחרוזת, שהיא אובייקט שאינו בר-שינוי (immutable) , את ה**רשימה אפשר לשנות** לאחר הגדרתה .

אפשר להוסיף לה איברים, לשנות אותם ואף למחוק אותם. אובייקט בר-שינוי נקרא **mutable**.

הוספת איבר ברשימה

הוספת איבר לרשימה מתבצעת באמצעות המתודה append או באמצעות **+=**.

לדוגמה, אם נגלה כוכב לכת חדש במערכת השמש, נוכל להוסיפו לרשימה באמצעות **append**:

```
>>> solar_system
["Mercury", "Venus", "Earth", "Mars", "Jupiter",
"Saturn", "Uranus", "Neptune"]
>>> solar_system.append("Novo")
>>> solar_system

['Mercury', 'Venus', 'Earth', 'Mars', 'Jupiter',
'Saturn', 'Uranus', 'Neptune', 'Novo']
```

או לחילופין הוספת איבר לרשימה באמצעות+= :

```
>> solar_system += ["Novo"]
```

באופן דומה, בעזרת האופרטור =+ אפשר לשרשר לרשימה גם רשימה שמכילה **יותר מאיבר אחד**.

שינוי איבר ברשימה

בנוסף להוספת איברים, ברשימה אפשר **לשנות** איברים קיימים, פעולה שאינה חוקית בעבודה עם מחרוזות .
לדוגמה, נשנה את שמו של כוכב הלכת החדש ברשימה:

```
>>> solar_system[8] = "De Novo"
>>> solar_system

['Mercury', 'Venus', 'Earth', 'Mars', 'Jupiter',
'Saturn', 'Uranus', 'Neptune', 'De Novo']
```

פרק 6 – רשימות

מחיקת איבר מרשימה

לעיתים נרצה למחוק איברים מרשימה .**מחיקת איבר** מרשימה יכולה להתבצע באמצעות המתודה **remove.** בסוגריים נכתוב את הערך אותו אנו מעוניינים למחוק .

שימו לב, רק ההופעה הראשונה של הערך תימחק מהרשימה.

דוגמה למחיקת כוכב הלכת החדש שיצרנו (De novo) מהרשימה solar_system :

```
>>> solar_system
['Mercury', 'Venus', 'Earth', 'Mars', 'Jupiter',
'Saturn', 'Uranus', 'Neptune', 'De Novo', 'Sun']
>>> solar_system.remove("De Novo")
>>> solar_system

['Mercury', 'Venus', 'Earth', 'Mars', 'Jupiter',
'Saturn', 'Uranus', 'Neptune', 'Sun']
```

מחיקת איברים

בואו ננסה להריץ שוב את המתודה remove עבור כוכב הלכת De Novo:

```
>>> solar_system.remove("De Novo")

Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: list.remove(x): x not in list
```

להזכירכם, כוכב הלכת De Novo כבר נמחק מהרשימה ולכן אינו מופיע בה. כשאנו מנסים למחוק ערך שאינו קיים אנו מקבלים הודעת שגיאה מסוג **ValueError**.
אנחנו עלולים להיתקל בבעיה זו לדוגמה כאשר נרצה למחוק את כל המופעים של ערך מסוים מרשימה, בלי לדעת מראש כמה פעמים הוא מופיע. פתרון אפשרי הוא לבדוק האם ערך **קיים** ברשימה לפני שמנסים למחוק אותו.

בדיקה האם איבר קיים ברשימה

בדומה למחרוזות, אפשר להשתמש **באופרטור in** גם עבור רשימות, בכדי לבדוק האם איבר כלשהו **קיים** ברשימה .

```
>>> solar_system
['Mercury', 'Venus', 'Earth', 'Mars', 'Jupiter',
'Saturn', 'Uranus', 'Neptune', 'Sun']
>>> "Earth" in solar_system

True
>>> "Pluto" in solar_system

False
```

פרק 6 – רשימות

פעולות שימושיות נוספות עם רשימות

ישנן מתודות ופונקציות רבות בעבודה עם רשימות. נכיר כמה מהן כאן, וכמובן שתגלו פעולות נוספות בהמשך בכוחות עצמכם.

לפניכם רשימה שנקראת temperatures. הרשימה מכילה אובייקטים מטיפוס int המציינים את הטמפרטורה הממוצעת במעלות פרנהייט בכוכבי הלכת במערכת השמש.

```
>>> temperatures = [-180, 800, -10, 60, -100, -180, 880,
-400]
```

ניעזר ברשימה זו כדי להכיר דוגמאות למתודות ולפונקציות שימושיות.

מתודה	תיאור	דוגמת שימוש במתודה	פלט מקריאת למתודה
Reverse	הופכת את סדר האיברים ברשימה, מהסוף להתחלה	temperatures.reverse()	[-400, 880, -180, -100, 60, -10, 800, -180]
sort	ממיינת את האיברים ברשימה, מהקטן לגדול	temperatures.sort()	[-400, -180, -180, -100, -10, 60, 800, 880]
count	מחזירה כמה פעמים מופיע ברשימה ערך כלשהו (שמועבר כפרמטר)	temperatures.count(-180)	2
index	מחזירה את המיקום הראשון ברשימה של ערך כלשהו (שמועבר כפרמטר)	temperatures.index(-100)	4

שימו לב שפעולות כמו reverse ו- sort **משנות** את הרשימה עצמה. זאת בניגוד לפעולות על מחרוזות (כגון upper) שמחזירות את תוצאת הפעולה אך אינן משנות את המחרוזת המקורית.

דוגמאות ל**פונקציות שימושיות**:

להזכירכם, הרשימה temperatures מכילה טמפרטורות בכוכבי הלכת במערכת השמש:

```
>>> temperatures = [-180, 800, -10, 60, -100, -180, 880,
                    -400]
```

פונקציה	תיאור	דוגמת שימוש בפונקציה	פלט מקריאה לפונקציה
max	מחזירה את האיבר הגדול ביותר ברשימה	max(temperatures)	880
min	מחזירה את האיבר הקטן ביותר ברשימה	min(temperatures)	-400
sum	מחזירה את הסכום של איברי הרשימה	sum(temperatures)	870
sorted	מחזירה רשימה חדשה בצורה ממוינת (לא משנה את הרשימה הקיימת)	sorted(temperatures)	[-400, -180, -180, -100, -10, 60, 800, 880]

פונקציה sorted עם מפתח:(key)

לפניכם רשימה עם איברים מטיפוס מחרוזת:

```
>>> my_list = ["A", "c", "b", "x", "Z", "i", "G", "J"]
```

נוכל למיין את הרשימה בעזרת הפונקציה sorted :

```
>>> sorted(my_list)
['A', 'G', 'J', 'Z', 'b', 'c', 'i', 'x']
```

שימו לב, הפונקציה sorted יכולה לקבל כפרמטר נוסף את ה**מפתח** (key) למיון .

בתור מפתח אנו יכולים לקבוע כל פונקציה שנרצה. לדוגמה, נניח שאנו רוצים למיין את הרשימה my_list כך שלא תבדיל בין אותיות קטנות לאותיות גדולות .

המשמעות היא שבמיון האות הקטנה b תופיע אחרי האות הגדולה A ולפני האות הגדולה G.

ניעזר במתודה str.lower שמחזירה עותק של המחרוזת באותיות קטנות ונעביר אותה כמפתח לפונקציה sorted :

```
>>> sorted(my_list, key=str.lower)
['A', 'b', 'c', 'G', 'i', 'J', 'x', 'Z']
```

בתיעוד של פייתון תוכלו ללמוד על פונקציות נוספות שמקבלות מפתח (או פרמטרים אופציונליים אחרים). לדוגמה, מקריאת הפירוט על הפונקציות min ו - max נלמד שגם הן יכולות לקבל מפתח כפרמטר:

max(arg1, arg2, *args[, key])

min(arg1, arg2, *args[, key])

באופן דומה, תוכלו להעביר כמפתח גם פונקציה שמימשתם בעצמכם בתוכנית. כדאי לכם לנסות זאת בעצמכם!

פרק 7 – לולאות

שימוש בלולאת while

כדי להכיר את הלולאה הראשונה, דמיינו דוב שנכנס לתרדמת חורף. הוא ישן כל עוד מזג האוויר חורפי ומתעורר רק כאשר מזג האוויר משתנה והטמפרטורות עולות .

כיצד נוכל לתאר בצורה פורמלית את פעולת הדוב? למעשה, הדוב שרוי בפעולת שינה אשר ממשיכה (בלולאה) כל עוד מתקיים (התנאי) מזג אוויר חורפי. הוא יוכל לצאת ממצבו רק כאשר ישתנה מזג האוויר.

```
כל-עוד מזג_אוויר_חורפי:
    דוב.לישון()
```

כמו בדוגמת הדוב, לפעמים נרצה לחזור על פעולה או פעולות **כל עוד** מתקיים תנאי כלשהו, כלומר מספר פעמים ש**אינו בהכרח קבוע** מראש .**הלולאה while** מאפשרת לבצע זאת בקלות.

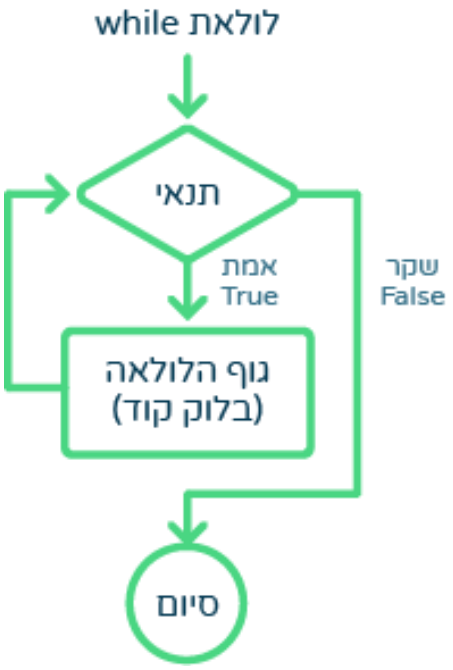
כיצד כותבים לולאת while בפייתון

לאחר המילה while יש לכתוב ביטוי תנאי ולאחריו נקודתיים. הפעולות לביצוע כתובות כבלוק מוזח.

כל סבב של הלולאה מכונה **איטרציה** (iteration). בכל איטרציה של הלולאה פייתון בודקת האם ה**תנאי** מתקיים.

אם לא ,כלומר הביטוי שבתנאי הוא שקר (False) , הלולאה מסתיימת.

אם כן ,כלומר הביטוי שבתנאי הוא אמת (True) , פייתון עוברת לביצוע הפקודות בבלוק הקוד המוזח, ולאחר מכן חוזרת לבדיקת התנאי באיטרציה נוספת.



פרק 7 – לולאות

בואו נראה כיצד אפשר להדפיס את המספרים מ-0 עד 4 בעזרת לולאת while:

```
>>> i = 0
>>> while i < 5:
        print(i)
        i += 1

0
1
2
3
4
```

לולאת while אינסופית

הפקודות בלולאת while מתבצעות שוב ושוב עד שהתנאי שהוגדר אינו מתקיים. לכן, אם התנאי מתקיים **תמיד** נוצרת **לולאה אינסופית**.

דוגמה לשימוש בלולאה אינסופית יכולה להיות בעבודה עם שרתים, כאשר מעוניינים להמשיך את העבודה באופן קבוע:

```
while True:
    Pass    # <Keep on working!>
```

אבל, מומלץ להיזהר מיצירת לולאה אינסופית בטעות. לעיתים היא נוצרת ללא כוונה, עקב כשל לוגי או חוסר תשומת לב.

בקרת זרימה בלולאות

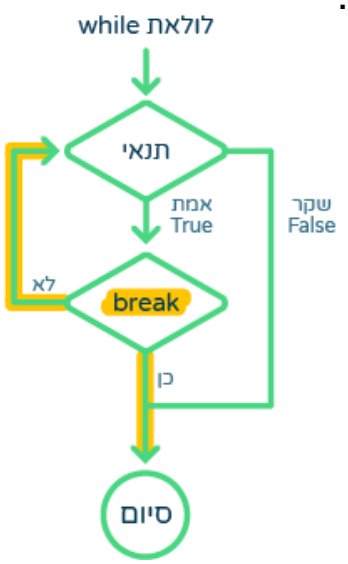
לפעמים נרצה **לצאת מלולאה** או **מאיטרציה מסוימת** בלולאה כאשר מתקיים תנאי אחר מזה שהוגדר בהתחלה .

לצורך כך נכיר שתי פקודות שמאפשרות בקרת זרימה עבור בלוקים של לולאות: הפקודה **break** והפקודה **continue**. פקודות אלה משמשות בלולאת while או בלולאת for .

פרק 7 – לולאות

הפקודה break

כאשר נרצה לצאת מיידית מהבלוק של הלולאה (במקרה זה לולאת while) , ללא איטרציות נוספות, נשתמש בפקודה **break**.

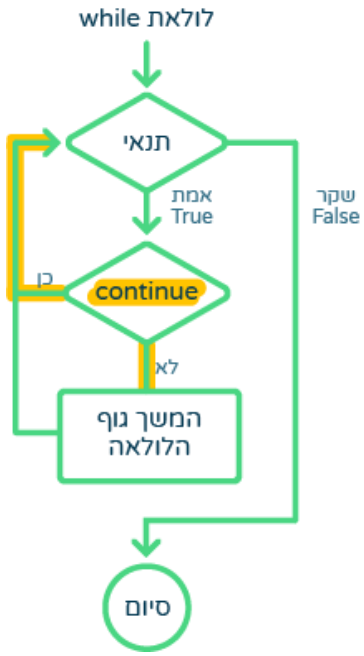



```
while i < 10:
    i += 1
    if i % 2 == 0:
        break
    print(i)

1
```

הפקודה continue

כאשר נרצה לעצור מייד את האיטרציה הנוכחית של הלולאה ולהתחיל את האיטרציה הבאה נשתמש בפקודה **continue**.



דוגמה לשימוש בפקודה continue בלולאת while :

```
i = 0
while i < 10:
    i += 1
    if i % 2 == 0:
        continue
    print(i)

1
3
5
7
9
```

שימוש בלולאות for

לפני שנתחיל, נגדיר את רשימת המחרוזות הבאה, המכילה שמות בעלי חיים:

```
>>> animals = ["Grizzly Bear", "Siberian Husky",
"Reindeer"]
```

כיצד נוכל להדפיס כל אחד מהאיברים ברשימה ?

אפשרות אחת להדפסת האיברים היא בעזרת המשתנה animal שיציבע בכל פעם לאיבר אחר, באופן הבא:

```
>>> animal = animals[0]
>>> print(animal)
Grizzly Bear
>>> animal = animals[1]
>>> print(animal)
Siberian Husky
>>> animal = animals[2]
>>> print(animal)
Reindeer
```

ודאי הבחנתם בכך שהשורות חוזרות על עצמן שוב ושוב, ואפשר לייצג אותן באופן כללי כך (כאשר בכל פעם x מייצג ערך עוקב בין 0 ל-2) :

```
>>> animal = animals[x]
>>> print(animal)
```

נוכל לחסוך את החזרה המיותרת על שורות הקוד ולהדפיס את שמות בעלי החיים **בלולאה** .

נתאר בצורה פורמלית את מה שהלולאה שלנו תעשה:

עבור כל חיה בתוך הרשימה חיות:

הדפס (חיה)

הלולאה for חוסכת מאיתנו כתיבה מיותרת, כיוון שהיא מבצעת את אותו בלוק של קוד פעם אחת עבור **כל** איבר המופיע ברצף (כבר למדנו שרשימות ומחרוזות הן רצפים).

כיצד כותבים לולאת for בפייתון

לאחר המילה for מגדירים משתנה בשם כלשהו (כל שם שנרצה), לאחריו מגיע האופרטור in , ובסיום אובייקט רצף כלשהו (לדוגמה רשימה) מלווה בנקודתיים. הפעולות לביצוע כתובות כבלוק מוזח .

צורת הכתיבה הכללית של לולאת for :

```
for <var> in <sequence>:
  <do something>
```

דוגמה לשימוש בלולאת for להדפסת שמות בעלי החיים מהרשימה animals :

```
animals = ["Grizzly Bear", "Siberian Husky", "Reindeer"]
for animal in animals:
   print(animal)

Grizzly Bear
Siberian Husky
Reindeer
```

פרק 7 – לולאות

בדוגמה לעיל עוברים בלולאה על האיברים ברשימה animals ומדפיסים כל אחד מהם. בשורה 2 אנו מגדירים משתנה שנקרא animal. בכל איטרציה של לולאת ה - for המשתנה animal מצביע לאיבר אחר (עוקב) מתוך הרשימה animals , ואז הערך של האיבר הזה מודפס.

שימו לב ש**האופרטור in** הוא חלק בלתי נפרד מהגדרת לולאת for בפייתון .

בכל איטרציה האופרטור in קושר באופן זמני את המשתנה שהגדרנו לאובייקט מן הרצף. זו מעין פעולת השמה שחוזרת בכל איטרציה.

חשוב לציין כי **בסיום ריצת הלולאה** המשתנה החדש שהוגדר (בדוגמה זו המשתנה animal) ממשיך להתקיים. ערך המשתנה נשאר מן האיטרציה האחרונה של הרצת הלולאה.

כאמור, גם מחרוזות הן רצפים. לכן, כפי שעברנו בלולאה על איברים ברשימה, אפשר לרוץ בעזרת לולאת for גם על תווים במחרוזת .

דוגמה לשימוש בלולאת for להדפסת תווים במחרוזת:

```
animal = "Reindeer"
for letter in animal:
  print(letter.upper())

R
E
I
N
D
E
E
R
```

רשימה בתוך רשימה

שימוש נחמד בלולאת for היא במקרה שיש לנו רשימה בתוך רשימה אחרת.

ניעזר ברשימה animal_sounds ובה רשימות של בעלי חיים והקולות שהם מפיקים:

```
>>> animal_sounds = [["bear", "roar"], ["dog", "bark"],
["deer", "bellow"]]
```

בעזרת לולאת for אפשר להציג בצורה נוחה את בעלי החיים יחד עם הקולות שהם מפיקים:

```
>>> animal_sounds = [["bear", "roar"], ["dog", "bark"],
["deer", "bellow"]]
>>> for item in animal_sounds:
        print("The sound a " + item[0] + " makes:\t" +
item[1])


The sound a bear makes: roar
The sound a dog makes: bark
The sound a deer makes: bellow
```

לולאה שרצה מספר מסוים של פעמים

לעיתים נרצה להריץ בלוק קוד מספר פעמים מוגדר, ולולאת for מאפשרת לעשות זאת בקלות .

לצורך כך, בואו נשווה בין דרכים שונות להדפיס את המספרים מ-0 עד 4, כלומר להריץ חמש פקודות הדפסה עבור משתנה.

דרך אחת שלמדנו היא באמצעות לולאת while שרצה 5 פעמים:

```
>>> num = 0
>>> while num < 5:
        print(num)
        num += 1

0
1
2
3
4
```

פרק 7 – לולאות

דרך אחרת לעשות בדיוק אותו דבר, אבל עם קוד קצר יותר, היא באמצעות לולאת ,for תוך שימוש ברשימה המכילה את המספרים 0-4:

```
>>> nums = [0,1,2,3,4]
>>> for num in nums:
        print num
0
1
2
3
4
```

דרך נוספת ואלגנטית, אשר חוסכת לנו לייצר בכל פעם את רשימת המספרים nums , היא בעזרת הפונקציה המובנית שנקראת **range.** הפונקציה range יוצרת התקדמות אריתמטית .

```
>>> for num in range(5):
    print(num)

0
1
2
3
```

הפונקציה range(5) מייצרת חמישה ערכים, בין 0 ל-5, לא כולל 5.

מבנה כללי של הפונקציה range

```
# The basic range() function
range(stop)

# The function range() with more parameters
range(start, stop[, step])
```

פרק 7 – לולאות

שימו לב שהפונקציה מקבלת פרמטרים שמאפשרים ליצור סדרות מורכבות יותר:

- start - מציין את ערך ההתחלה (ערך ברירת המחדל הוא 0).
- stop - מציין את ערך הסיום (לא כולל). ערך זה **תמיד** נכלל בקריאה לפונקציה.
- step - מציין את מרווח הקפיצה בין איברים בסדרה (יכול להיות גם שלילי).

דוגמאות לשימוש בפונקציה range עם ארגומנטים שונים:

```
>>> for num in range(5, 10):
    print(num)

5
6
7
8
9
>>> for num in range(0, 10, 3):
    print(num)

0
3
6
9
```

פרק 8 – טיפוסים מתקדמים של מבני נתונים

מהו טאפל?

טאפל (רשומה, באנגלית **tuple**) הוא מבנה נתונים שמסוגל להכיל איברים מטיפוסים שונים.
בפרק זה נעמוד על אופיו ועל ההבדלים בינו לבין רשימה.

להזכירכם, רשימה מגדירים על ידי סוגריים מרובעים [ ]. לעומתה, טאפל מסמנים על ידי **סוגריים עגולים** ( ).

בכתיבת סוגריים עגולים אפשר להגדיר טאפל , אפילו ללא ערכים .

```
>>> my_first_tuple = ()
>>> type(my_first_tuple)
<class 'tuple'>
>>> print(my_first_tuple)
()
```

כדי להגדיר טאפל המכיל **מספר איברים** (שניים או יותר), נפריד בין ערכי האיברים באמצעות פסיקים .

דוגמה להגדרת טאפל המכיל 3 איברים:

```
>>> friendly_tuple = (2.5, 3, "banana")
>>> friendly_tuple
(2.5, 3, 'banana')
```

שימו לב שאפשר להגדיר טאפל גם באמצעות ערכים שמופרדים בפסיק **ללא סוגריים** .במקרה כזה, פייתון תארוז את האיברים כטאפל בעצמה:

```
>>> sophisticated_tuple = 2, "tomato"
>>> sophisticated_tuple
(2, 'tomato')
```

יצירת טפאל עם איבר יחיד נוסיף **פסיק לאחר האיבר**

```
>>> one_value_tuple = (20,)
>>> one_value_tuple
(20,)
>>> type(one_value_tuple)
<class 'tuple'>
```

פרק 8 – טיפוסים מתקדמים של מבני נתונים

דמיון ושוני בין טפאל לבין רשימה

הטאפל והרשימה הם שני מבני נתונים הדומים זה לזו – באפשרותם להחזיק מספר איברים מטיפוסים שונים, ובנוסף, שניהם רצפים ולכן ניגש אליהם ונעבוד איתם באמצעות מיקומים (אינדקסים). המשמעות היא שפעולות רבות שהכרנו עבור רשימות (כגון פעולות גישה, איטרציות בלולאה וכדומה) משמשות גם עבור טאפלים.

דוגמאות לפעולות שימושיות עבור הטיפוס טאפל:

```
>>> milk_info = ("lactose", "11/08/2022", "dairy cows",
"135.6 liters", 300)
>>> len(milk_info)
5

>>> milk_info[0]
'lactose'

>>> milk_info[2:]
('dairy cows', '135.6 liters', 300)

>>> str(milk_info[4])+ " "+milk_info[2]
'300 dairy cows'

>>>for info in milk_info:
   print("-- " + str(info) + " --")

-- lactose --
-- 11/08/2022 --
-- dairy cows --
-- 135.6 liters --
-- 300 -
```

טאפל הוא immutable

כאן טמון **ההבדל המהותי בין טאפל לבין רשימה** :

רשימה היא טיפוס שניתן לשינוי (באנגלית mutable) , ואילו **טאפל הוא טיפוס שאינו ניתן לשינוי** (באנגלית **immutable**).

פרק 8 – טיפוסים מתקדמים של מבני נתונים

איברים ברשימה אפשר לשנות באמצעות השמה, הוספה, מחיקה ופעולות נוספות. לעומתם, אחרי הגדרת טאפל **לא קיימת אפשרות** לשנות את תוכנו, להוסיף איבר או למחוק איבר. אחרי הגדרת טאפל **קיימת אפשרות** רק לגשת אליו לצורך קריאה. להזכירכם, באופן דומה גם הטיפוס מחרוזת הוא אינו בר-שינוי.

אם כך, מהם **היתרונות של הטיפוס טאפל**?

- טאפל הוא מבנה נתונים **בטוח** כיוון שהוא מגן על נתונים שלא צריכים להשתנות. מומלץ להשתמש בטאפל כשאנו מעוניינים לוודא שמידע שאנו מאחסנים לא יעבור כל שינוי.
- טאפל הוא מבנה נתונים **חסכוני** יותר בהקצאת מקום בזיכרון מאשר רשימה. פייתון מקצה לטאפל זיכרון בדיוק בגודל הנדרש, בעוד שלרשימה מוקצה זיכרון נוסף שמאפשר לבצע בה שינויים (כדוגמת הוספת איברים).

בחירת מבנה נתונים

כיצד נחליט מתי להשתמש בטאפל עבור אחסון מידע ?

נבחן את **אופי המידע** שברשותנו.

על פי קונבנציה נעדיף להשתמש בטאפל על פני רשימה במקרים הבאים:

- המידע הנו **קבוע מראש** .כלומר, מספר האיברים וערכיהם אינם עתידים להשתנות.
- קיימת חשיבות ל**מיקום** של האיברים בתוך מבנה הנתונים, כלומר לסדר בו הם מאוחסנים.

דוגמה לשימוש בטאפל עבור מידע שאינו עתיד להשתנות ושקיימת בו חשיבות לסדר האיברים היא הגדרת נקודה מסוימת על ידי הקורדינאטות x ו-y:

```
>>> point = (5, 2)
```

האיברים בטאפל point מייצגים ביחד מושג שנקרא "נקודה". הנקודה מורכבת משני איברים בדיוק, והסדר שלהם הוא בעל משמעות: האיבר במיקום אחת הוא קואורדינטת ה-x , והאיבר במיקום שתיים הוא קואורדינטת ה-y.



פרק 8 – טיפוסים מתקדמים של מבני נתונים

השמה באמצעות טאפל

בפייתון אפשר להשתמש בטאפלים לצורך ביצוע **פעולת השמה מרובה** .כלומר, זהו קיצור דרך שימושי ואלגנטי עבור השמת ערכים למספר משתנים בפעולה יחידה.

דוגמה להשמת שלושה ערכים לשלושה משתנים באמצעות טאפלים (tuple assignment):

```
>>> (food, type, calories) = ("bread", "whole wheat",
80)
>>> food
'bread'
>>> calories
80
```

בדוגמה זו נוצרו בטאפל השמאלי שלושה **משתנים** (כלומר מצביעים) ובטאפל הימני שלושה **ערכים** .בפועל, פייתון ביצעה **השמה** בין כל זוג איברים מתאימים כגון: הצבעה של המשתנה שנקרא food על הערך "bread".

לפעולת ההשמה באמצעות טאפלים קיימים שימושים רבים. דוגמה נפוצה, בה נתקלנו ביחידה על משתנים, היא החלפה בין ערכים של שני משתנים ללא צורך במשתנה זמני:

```
# exchange gifts using tuples
>>> my_gift, your_gift = your_gift, my_gift
```

החזרת ערכים מפונקציה

ביחידה שעוסקת בפונקציות למדנו שפונקציה יכולה להחזיר יותר מערך אחד. נראה כיצד מבנה הנתונים טאפל משמש ל **פעולת החזרת ערכים** זו.

ניזכר בקטע קוד שמדגים הגדרת פונקציה שמחזירה שני ערכים:

```
>>> def sum_and_avg(a, b):
        sum_num = a + b
        avg_num = (a + b) / 2
        return sum_num, avg_num
```

פרק 8 – טיפוסים מתקדמים של מבני נתונים

בפועל, הפונקציה sum_and_avg מחזירה את שני הערכים sum_num ו - avg_num בתור **טאפל** .זכרו, אין הכרח לכתוב את הטאפל עם סוגריים.

דוגמה לקריאה לפונקציה sum_and_avg ושימוש בפונקציה type לבדיקת הטיפוס שמוחזר ממנה:

```
>>> type(sum_and_avg(3, 6))
<class 'tuple'>
```

דוגמה לקריאה לפונקציה sum_and_avg והשמת ערכי ההחזרה לשני משתנים:

```
>>> x, y = sum_and_avg(3, 6)
>>> x
9
>>> y
4.5
```

יצירת תבנית למחרוזת

מקרה נפוץ נוסף בו משתמשים במבנה הנתונים טאפל הוא עבור **יצירת תבנית למחרוזת** (באנגלית - **string formatting**)

לפנינו שני משתנים המתארים פריטים שנמצאים בתוך סלסילת פיקניק:

```
>>> fruit = "grapes"
>>> drink = "wine"
```

אם נרצה להדפיס תיאור של תכולת הסלסילה, נוכל לבצע זאת בשתי דרכים:

דרך אחת, כפי שלמדנו במחרוזות, היא באמצעות **שרשור**:

פרק 8 – טיפוסים מתקדמים של מבני נתונים

```
>>> "The basket contains " + fruit + " and " + drink + "."
'The basket contains grapes and wine.'
```

באפשרות זו השתמשנו באופרטור '+' על מנת לשרשר בין מחרוזות שהקלדנו בפקודת ההדפסה, לבין מחרוזות שמגיעות מתוך משתנים (fruit ו-drink)

דרך נוספת לביצוע פעולת ההדפסה היא באמצעות **string formatting**.

בשיטה זו אנו משתמשים בתו '%' ליצירת תבנית להדפסת מחרוזות. נכתוב **%s** בכל פעם שנרצה לשבץ מחרוזת המגיעה מתוך משתנה. בסיום פקודת ההדפסה נעביר את המשתנים אותם אנו רוצים לשבץ כ**טאפל המלווה בסימן %** , תוך הקפדה על סדר האיברים.

```
>>> "The basket contains %s and %s." % (fruit, drink)
The basket contains grapes and wine.
```

שימו לב שבתבנית זו יצרנו **הפרדה ברורה** בין המחרוזת המוקלדת בפקודת ההדפסה לבין המשתנים. צורה זו נוחה הרבה יותר עבור תחזוקת הקוד, למשל שינוי המשתנים.

להזכירכם, בשיטת השרשור ('+') אפשר לשרשר מחרוזות בלבד. אם נרצה לשרשר מספר לדוגמה, נצטרך להמירו לטיפוס מחרוזת. **אם לא** נבצע המרה מפורשת של המספר למחרוזת - נקבל הודעת **שגיאה** מסוג TypeError .

לעומת זאת, בשיטת string formatting אפשר לשבץ במחרוזת גם **טיפוסים נוספים**, שאינם מחרוזת. לדוגמה, בכדי להדפיס **מספר שלם** (int) , נכתוב **%d** במקום בו נרצה לשבצו. אפשרויות נוספות תמצאו בהמשך

דוגמה להדפסת מספר ומחרוזת בשיטת string formatting :

```
>>> bottles_num, drink = 3, "wine"
>>> "The basket contains %d bottles of %s." %
(bottles_num, drink)
The basket contains 3 bottles of wine.
```

בדוגמה לעיל העברנו בתוך פקודת ההדפסה טאפל שמורכב מכמה משתנים. לחלופין, אפשר להעביר משתנה שמצביע על טאפל שמורכב מן האיברים בסדר הנכון.

דוגמה להדפסת מספר ומחרוזת באמצעות הגדרת המשתנה basket_content מטיפוס טאפל:

```
>>> basket_content = 3, "wine"
>>> "The basket contains %d bottles of %s." %
basket_content
The basket contains 3 bottles of wine.
```

ראו כיצד מחרוזות מורכבות, שמכילות ערכים של משתנים, הפכו בקלות לקריאות למשתמש!

קיימות וריאציות רבות ואפשרויות אינספור לביצוע string formatting במחרוזות.

מקודם למדנו לבצע פעולות בשיטה הישנה ("old style") באמצעות שימוש בסימן "%".
בגרסאות פייתון 2.6 והלאה קיימת אפשרות לבצע את הפעולה גם באמצעות מתודה הנקראת **format**.

בנוסף, ישנן אפשרויות רבות לעיצוב פקודות ההדפסה, בשתי השיטות.

פרק 8 – טיפוסים מתקדמים של מבני נתונים

מהו מילון?

בחיי היומיום אנו נעזרים במילוני שפה: במילון עברי-עברי נוכל לחפש מילה בעברית שאיננו מכירים, ולמצוא ערך שהוא הפירוש שלה. במילון אנגלי-עברי נוכל לחפש מילה באנגלית ולמצוא ערך שהוא התרגום שלה לעברית.

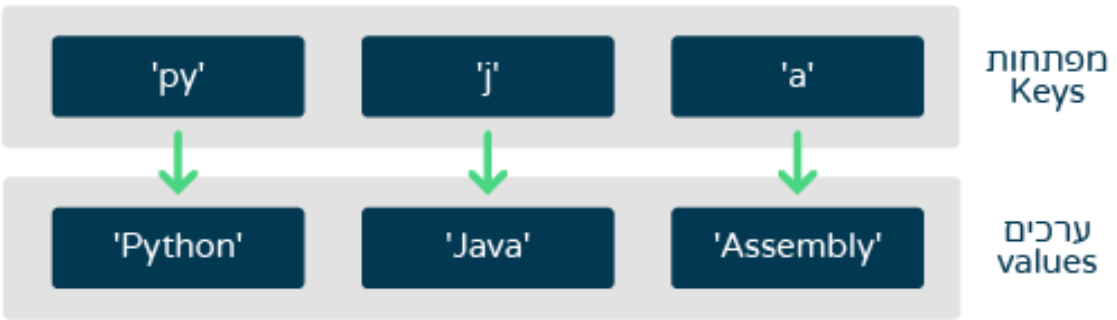
כעת נלמד על **המילון הפייתוני** (באנגלית ,dictionary או בקיצור dict): מבנה נתונים שקיים בשפת פייתון, שעובד באופן דומה למילונים אותם הזכרנו. המילה אותה אנו מחפשים במילון היא **המפתח** ,והפירוש או התרגום של המילה הוא **הערך** .למעשה, כל איבר במילון הוא צמד של מפתח (key) וערך (value).

המילון מכיל ערכים בדומה לרשימה, אך בניגוד אליה הערכים שבו מסודרים על פי מפתחות ולא כאיברים בסדר רציף. כאשר נפנה למילון נציין מפתח ונקבל בעבורו ערך. היתרון בכך הוא שבמקום להשתמש במיקומים (כלומר לציין אינדקס, שהוא מספר שלא בהכרח אומר לנו משהו), נוכל להשתמש במפתח שהוא מידע בעל משמעות.

גישה לאיברים ברשימה באמצעות מיקומים



גישה לאיברים במילון באמצעות מפתחות



פרק 8 – טיפוסים מתקדמים של מבני נתונים

יצירת מילון?

מילון מוגדר באמצעות **סוגריים מסולסלים** { }.

אפשר ליצור מילון ריק, כלומר מילון ללא מפתחות וערכים, בשתי דרכים :

דרך אחת ליצירת מילון היא על ידי כתיבת סוגריים מסולסלים:

```
>>> new_dict = {}
>>> type(new_dict)
<class 'dict'>
>>> new_dict
{}
```

דרך נוספת ליצירת מילון היא באמצעות הפונקציה dict() :

```
>>> another_dict = dict()
>>> type(another_dict)
<class 'dict'>
>>> another_dict
{}
```

אפשר לאתחל מילון עם **ערכים התחלתיים** :נכתוב סוגריים מסולסלים ובתוכם איברים שהם צמדים הבנויים ממפתח, לאחריו נקודתיים ורווח, והערך המתאים למפתח. בין איברים נפריד בפסיקים .

מבנה של צמד במילון המורכב ממפתח וערך:

```
key: value
```

דוגמה ליצירת מילון שבו המפתח (key) מייצג פעולת עריכה במחשב, והערך (value) מייצג את מקשי הקיצור במקלדת שמשמשים לביצוע הפעולה:

```
>>> keyboard_shortcuts = {"Copy": "Ctrl + C", "Paste":
"Ctrl + V", "Undo": "Ctrl + Z"}
{'Copy': 'Ctrl + C', 'Paste': 'Ctrl + V', 'Undo': 'Ctrl +
Z'}
```

פרק 8 – טיפוסים מתקדמים של מבני נתונים

גישה לערך באמצעות מפתח

אפשר **לגשת לערכים במילון** באמצעות ציון המפתח שלהם בתוך סוגריים מרובעים [ ], בהתאם למבנה הבא:

```
dict[key]
```

דוגמה לשליפת ערך מתוך מילון:

```
>>> keyboard_shortcuts["Paste"]
'Ctrl + V'
```

שימו לב ש**בלתי אפשרי לגשת למפתחות** באמצעות הערכים שלהם.

KeyError

היזכרו במילון keyboard_shortcuts שיצרנו. מה לדעתכם יקרה אם נריץ את השורה הבאה?

```
>>> keyboard_shortcuts
{'Copy': 'Ctrl + C', 'Paste': 'Ctrl + V', 'Undo': 'Ctrl + Z'}
>>> keyboard_shortcuts["Cut"]
Traceback (most recent call last):
   File "<stdin>", line 1, in <module>
KeyError: 'Cut'
```

לא מאוד מפתיע… קיבלנו הודעת שגיאה מסוג **KeyError** כיוון שהמפתח שנקרא "Cut" אינו קיים במילון.

כיצד נימנע מקריסה של תוכנית עקב קריאה למפתח שאינו קיים?

נוכל להקדים ולבדוק האם המפתח קיים, לדוגמה:

```
>>> if "Cut" in keyboard_shortcuts:
    print(True)
```

פרק 8 – טיפוסים מתקדמים של מבני נתונים

שינוי מילון - מילון הוא mutable

המילון הוא מבנה נתונים שניתן לשינוי, כלומר **mutable.** המשמעות היא שאפשר לבצע בו פעולות הוספה, עדכון, מחיקה ועוד.

הוספת איבר למילון

הוספת איבר למילון מתבצעת במבנה הבא:

```
dict[key] = value
```

דוגמה להוספת הפעולה "Cut" ומקשי הקיצור שלה במקלדת למילון:

```
>>> keyboard_shortcuts
{'Copy': 'Ctrl + C', 'Paste': 'Ctrl + V', 'Undo': 'Ctrl + Z'}
>>> keyboard_shortcuts["Cut"] = "Ctrl + X"
>>> keyboard_shortcuts
{'Copy': 'Ctrl + C', 'Paste': 'Ctrl + V', 'Undo': 'Ctrl + Z', 'Cut': 'Ctrl + X'}
```

עדכון איבר במילון

מתכנת כלשהו מעדיף את מקשי הקיצור Ctrl + Insert עבור פעולת ההעתקה ("Copy"). מה לדעתכם יקרה אם ינסה לעדכן את מקשי הקיצור עבור הפעולה באמצעות הפקודה שלמדנו זה עתה (להוספת איבר)? במילים אחרות, מה יקרה אם ננסה להוסיף מפתח אשר קיים כבר במילון?

```
>>> keyboard_shortcuts
{'Copy': 'Ctrl + C', 'Paste': 'Ctrl + V', 'Undo': 'Ctrl + Z', 'Cut': 'Ctrl + X'}
>>> keyboard_shortcuts["Copy"] = "Ctrl + Insert"
>>> keyboard_shortcuts
{'Copy': 'Ctrl + Insert', 'Paste': 'Ctrl + V', 'Undo': 'Ctrl + Z', 'Cut': 'Ctrl + X'}
```

פרק 8 – טיפוסים מתקדמים של מבני נתונים

כפי שראיתם, בפייתון כל מפתח הוא **ייחודי** ואין אפשרות לחזור על מפתח קיים. במקרה זה, פייתון לא זורקת הודעת שגיאה אלא **מעדכנת** ערך חדש למפתח.

טיפוסי מפתחות וערכים

עד כה ראינו במילון מפתחות וערכים שכולם מטיפוס מחרוזת.

ערכים במילון יכולים להיות **מטיפוסים שונים** ,כגון מחרוזת, רשימה, טאפל , מספר שלם ועוד .
דוגמה למילון keyboard_info שמכיל ערכים מטיפוסים שונים:

```
>>> keyboard_info = {"Company": "Logitech", "Number of
keys": 101, "Are emoji included?": True, "Language":
"Hebrew"}
```

מחיקה של איבר ממילון

```
>>> keyboard_info
{'Company': 'Logitech', 'Number of keys': 101, 'Are emoji
included?': True, 'Language': 'Hebrew'}
>>> del keyboard_info["Number of keys"]
>>> keyboard_info
{'Company': 'Logitech', 'Are emoji included?': True,
'Language': 'Hebrew'}
```

דרך אחרת היא באמצעות **המתודה ()pop** המתודה מחזירה את הערך המתאים למפתח שהועבר כפרמטר ולאחר מכן מסירה את האיבר:

```
>>> keyboard_info.pop("Company")
'Logitech'
>>> keyboard_info
{'Are emoji included?': True, 'Language': 'Hebrew'}
```

כשעובדים עם מילון קיימת אפשרות **הצגה** (view) של המפתחות ושל הערכים של המילון בדרך נוחה וחסכונית (מבחינת מקום בזיכרון המחשב).

פרק 8 – טיפוסים מתקדמים של מבני נתונים

הצגת מפתחות או ערכים

בכדי להציג את כל המפתחות **או** את כל הערכים במילון נוכל להשתמש במתודה keys() ובמתודה values() בהתאמה.

```
>>> keyboard_info
{'Are emoji included?': True, 'Language': 'Hebrew'}
>>> keyboard_info.keys()
dict_keys(['Are emoji included?', 'Language'])
```

```
>>> keyboard_info.values()
dict_values([True, 'Hebrew'])
```

כיוון שמילון הוא אוסף של איברים, אפשר להשתמש במתודות לעיל ולעבור על המפתחות או על הערכים באמצעות לולאת for.

דוגמה להדפסת כל המפתחות במילון ולהדפסת כל הערכים במילון בעזרת לולאת for:

```
>>> for key in keyboard_info.keys():
    print(key)

Are emoji included?
Language

>>> for value in keyboard_info.values():
    print(value)

True
Hebrew
```

פרק 8 – טיפוסים מתקדמים של מבני נתונים

הצגת מפתחות וגם ערכים

בכדי להציג את כל ה**צמדים** במילון: המפתחות **וגם** הערכים שלהם, נוכל להשתמש במתודה items() באופן זה כל צמד מיוצג כטאפל.

```
>>> keyboard_info
{'Are emoji included?': True, 'Language': 'Hebrew'}
>>> keyboard_info.items()
dict_items([('Are emoji included?', True), ('Language',
'Hebrew')])
```

בכדי להדפיס את כל הצמדים במילון בצורה מעוצבת, נוכל להיעזר בשיטת string formatting לדוגמה:

```
>>> for info in keyboard_info.items():
    print("%s -> %s" % info)
Are emoji included? -> True
Language -> Hebrew
```

הצגה של מילון

בכדי להציג מפתחות, ערכים וצמדים שלהם הכרנו את המתודות `keys`, `values` ו-`items`. האובייקטים המוחזרים ממתודות אלה נקראים בפייתון **`view objects`**. אובייקטים אלו מספקים תצוגה דינמית על המפתחות, הערכים והצמדים, כך שכל שינוי במילון משוקף גם באובייקטים.

פרק 9– קבצים

שימוש בקבצים עבור קלט ופלט

עד כה נהגנו לקבל לתוכנית קלט (input) שהוא הקלדה מהמשתמש, באמצעות הפקודה input . כאשר רצינו להציג פלט של התוכנית (output) עשינו זאת בעזרת הפקודה print.

דרך נוספת בעבודה עם קלט ופלט היא באמצעות **קבצים** המוכרים לנו מהשימוש היומיומי שלנו במחשב.
דוגמאות לקבצים הן: מסמכים, תמונות, קבצי מוזיקה, קבצי וידאו או קבצי הרצה. למעשה רוב התוכנות היום משתמשות בקבצים, לדוגמה: תוכנות לעריכת תמונות או וידאו, תוכנות אנטי-וירוס שמקבלות עדכוני תוכנה באמצעות קבצים שהן מורידות, דפדפנים ששומרים את היסטוריית הגלישה שלנו על המחשב ועוד.
קבצים מאפשרים לנו לקבל קלט ולהחזיר פלט בצורה נוחה, במיוחד כאשר מדובר ב**כמויות גדולות** של מידע.
בפרק הזה נלמד לעבוד עם **קבצי טקסט** אלה קבצים עם סיומת txt המכילים תווים קריאים כגון מספרים, אותיות וסמלים. נכיר דרכים לפתוח קבצי טקסט, לערוך אותם ולבצע עליהם פעולות נוספות.
דוגמה לקובץ טקסט שנקרא spells.txt ומכיל לחשי-קסמים מהסרט "הארי פוטר":"
שימו לב, קיים סוג נוסף של קבצים הנקרא **קבצים בינאריים** ,לדוגמה: קבצי תמונה או קבצי וורד.

פתיחת קובץ לקריאה:

עבודה עם קבצים בפייתון היא פשוטה למדי. בכדי לפתוח קובץ עלינו לקרוא לפונקציה המובנית **open()**

```
file_object = open(file, mode)
```

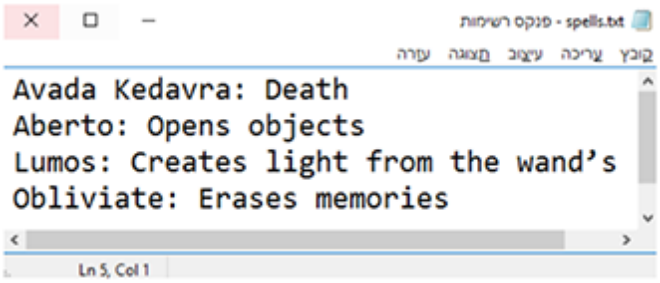
הפרמטרים שהפונקציה open() מקבלת הם:

- **file** - מחרוזת המייצגת את **נתיב הגישה לקובץ** איתו רוצים לעבוד.
- **mode** - מחרוזת המציינת את **סוג הגישה לקובץ** ,לדוגמה 'r' עבור קריאה של טקסט מהקובץ ו 'w'- עבור כתיבה של טקסט לקובץ.

הפונקציה open() מחזירה **אובייקט מטיפוס קובץ** , שמשמש לייצוג הקובץ איתו אנו מעוניינים לעבוד. בדוגמה לעיל המשתנה file_object מצביע עליו. לאחר פתיחת הקובץ נוכל לבצע פעולות שונות על האובייקט מטיפוס קובץ כגון: גישה לתוכן של הקובץ ועדכון שלו.

```
>>>spells_input_file= open("c:\harry_potter\spells.txt","r")
```

בדוגמה זו פתחנו את הקובץ שנקרא spells.txt בגישת קריאה בלבד, ונוצר אובייקט מטיפוס קובץ עליו מצביע המשתנה שנקרא spells_input_file.



שימו לב, אם הקובץ אינו קיים ביעד, פקודת פתיחת הקובץ **אינה** יוצרת קובץ בעצמה. לכן ודאו שהנתיב שאתם כותבים מאויית כראוי ומוביל לקובץ קיים.

התחילית r:

זוכרים מהם תווים מיוחדים בכתיבת מחרוזות? הביטו בדוגמה הבאה:

```
>>> input_file = open("c:\telephones.txt", "r")
```

כיוון שנתיב הקובץ מכיל את התו \ ומייד אחריו את התו t , פייתון מתייחסת אליהם יחד בתור התו המיוחד \t (טאב) ולא כאל מעבר לספרייה חדשה. הדבר עלול לגרום לבעיות לא צפויות.

כיצד אפשר להימנע מכך?

אם אתם רוצים לסמן לפייתון להתייחס למחרוזת (במקרה זה נתיב הקובץ) בדיוק כפי שהיא, ללא המרה לתווים מיוחדים, תוכלו להשתמש **בתחילית r** ולכתוב אותה **לפני המחרוזת.**

דוגמה לשימוש בתחילית r לצורך ייצוג הנתיב לקובץ כפי שהוא, ללא המרה לתווים מיוחדים:

```
>>> input_file = open(r"c:\telephones.txt", "r")
```

חשוב לציין שהתחילית r שימושית בעבודה עם מחרוזות **בהקשרים נוספים** ולא רק בעבודה עם קבצים.

פרק 9– קבצים

סגירת קובץ

מתי יסגר קובץ בפייתון?

כאשר פותחים קובץ באמצעות הפונקציה open(), פייתון מבקשת ממערכת ההפעלה לנעול את הקובץ לשימוש על ידי תוכנות אחרות. הנעילה חשובה בכדי למנוע כתיבה לאותו קובץ במקביל, לדוגמה. הקובץ נותר נעול עד שפייתון תשחרר אותו. אמנם הקובץ ישוחרר וייסגר בשלב מסוים, אך לא נפרט על המנגנון האחראי לכך. למרות שהקובץ עתיד להיסגר בזמן כלשהו, בסיום העבודה איתו בקוד מומלץ **לסגור אותו באופן יזום** בכדי למנוע מהתוכנית להתנהג בצורה בלתי צפויה כדוגמת אי שמירה של שינויים בקובץ.

סגירת קובץ נעשית באמצעות המתודה **close()**.

דוגמה לסגירה של קובץ באמצעות המתודה close(), לאחר שפתחנו אותו לקריאה:

```
>>> spells_input_file =
open("c:\harry_potter\spells.txt", "r")
>>> spells_input_file.close()
```

שימו לב, בין פתיחת הקובץ לסגירתו נוכל כמובן לבצע על הקובץ פעולות מגוונות כגון קריאה ממנו, כתיבה אליו ועוד. חלק מן הפעולות נכיר בהמשך הקורס ובחלקן תיתקלו בעצמכם בעתיד.

גם פייתון יודעת לעשות קסמים, והיא מאפשרת לנו לפתוח קבצים מבלי לדאוג לסגור אותם **באופן מפורש.**
נרמוז לפייתון שאנו מעוניינים לסגור את הקובץ באופן אוטומטי בסיום העבודה אם נשתמש ב**הצהרה with.**

עבודה עם ההצהרה with מומלצת כיוון שהיא משחררת את משאבי המערכת (כגון קבצים, אובייקטי סינכרון, חיבורי רשת ועוד), כפי שלמדנו על סגירת קובץ.
ההצהרה with פותחת בלוק של קוד בסיומו ייסגר הקובץ והיא נכתבת במבנה הבא:
ההצהרה **with** , אחריה קריאה לפונקציה open() עם הארגומנטים המתאימים, המילה **as** ולאחריה שם של משתנה שמצביע על הקובץ ולבסוף נקודתיים שפותחות בלוק מוזח.

```
with open(file,mode) as input_file:
    #do something with the file
```

כאשר כותבים קוד במבנה הזה, הקובץ יישאר פתוח עד לסיום ריצת הבלוק ,with ולאחר מכן ייסגר באופן אוטומטי.

כאמור, השימוש בהצהרה with מחליף את הצורך לסגור את הקובץ באמצעות הפונקציה close().

לדוגמה, במקום לכתוב את קטע הקוד שכולל סגירה מפורשת של הקובץ:

```
spells_input_file = open("c:\harry_potter\spells.txt",
"r")
# do something with the file
spells_input_file.close()
```

ניעזר בהצהרה with ונכתוב:

```
with open("c:\harry_potter\spells.txt",
"r") as spells_input_file:
# do something with the file
```

קריאת תוכן של קובץ

בפרויקטים מורכבים בפייתון אנו צפויים להידרש להתמודד עם מידע רב ממקורות שונים. לצורך כך נרצה לשמור, לנתח ולעבד את המידע כרצוננו. לפיכך, לאחר שפתחנו קובץ כלשהו, לרוב נתחיל ממעבר על תוכנו: נרצה לקרוא את כולו או חלקים ממנו.
נכיר שתי שיטות שימושיות ל**קריאת תוכן של קובץ** שפתחנו, שפועלות על אובייקטים מטיפוס קובץ. השיטה הראשונה - קריאת הקובץ כולו באמצעות המתודה ()read , והשיטה השנייה - מעבר על הקובץ שורה - שורה באמצעות לולאה. קיימות גם שיטות נוספות, ואפשר לבחור מתי להשתמש בכל אחת בהתאם לצרכי התוכנית שכותבים.

לצורך קריאת תוכן מקובץ טקסט, נתייחס לקובץ כאל **אוסף של שורות טקסט**, כאשר כל שורה היא מחרוזת שמסתיימת בסימון שורה חדשה .(\n) בפועל לא תמיד רואים סימון זה, אך בכל קובץ המכיל יותר משורה אחת, הסימון \n קיים בסופה של כל שורה.

בכדי לקרוא את **תוכן הקובץ כולו** ניעזר במתודה **()read**. המתודה מחזירה את תוכן הקובץ כולו כ**מחרוזת אחת** שכוללת \n בסוף כל שורה של הקובץ המקורי.

```
>>> spells_input_file =
open("c:\harry_potter\spells.txt", "r")
>>> spells = spells_input_file.read()
>>> spells
"Avada Kedavra: Death\nAberto: Opens objects\nLumos:
Creates light from the wand's tip\nObliviate: Erases
memories\n"
```

ודאי שמתם לב שהתו האחרון בכל שורה בקובץ הוא n\ המייצג ירידת שורה.
ניעזר בפונקציה print() כדי להדפיס את תוכן הקובץ כולל התייחסות לירידות השורה כאל תווים מיוחדים:

```
>>> spells_input_file =
open("c:\harry_potter\spells.txt", "r")
>>> spells = spells_input_file.read()
>>> print(spells)
Avada Kedavra: Death
Aberto: Opens objects
Lumos: Creates light from the wand's tip
Obliviate: Erases memories
```

פשוט וקל, נכון?

כעת חשבו:
מה החיסרון בקריאת תוכן של קובץ בעזרת המתודה ()read?

הדוגמה לעיל מכילה קובץ טקסט קצרצר ובו ארבע שורות בלבד. לעומת זאת, בעבודה עם קבצים גדולים, השמה של תוכן קובץ לתוך משתנה יחיד עלולה ליצור עומס בזיכרון ולהאט את ריצת התוכנית. לפיכך המתודה ()read אינה מתאימה לקריאת קבצים גדולים.

בעבודה עם קבצים גדולים מומלץ להיעזר בשיטה אחרת: מעבר על **שורה אחת בכל פעם** , באמצעות **לולאת for .**
בצורה זו נוכל גם **להדפיס** את הקובץ שורה אחר שורה:

```
>>> spells_input_file =
open("c:\harry_potter\spells.txt", "r")
>>> for line in spells_input_file
    print(line, end="")

Avada Kedavra: Death
Aberto: Opens objects
Lumos: Creates light from the wand's tip
Obliviate: Erases memories
```

פרק 9– קבצים

לסיכום פרק זה של עבודה עם קבצים נביט בתוכנית לדוגמה. התוכנית מדפיסה את הכשפים ה"ידידותיים" בקובץ spells.txt. כשפים "ידידותיים" הם כשפים שאינם כוללים את המילה "Death" (מוות):

```
spells_input_file = open("c:\harry_potter\spells.txt",
"r")
for line in spells_input_file:
    if "Death" not in line:
        print(line)
spells_input_file.close()
Aberto: Opens objects
Lumos: Creates light from the wand's tip
Obliviate: Erases memories
```

פירוט שלבי התוכנית:

- בשורה 1 הקובץ spells.txt **נפתח לקריאה.**
- בשורה 2 **עוברים על תוכן הקובץ** שורה אחר שורה באמצעות לולאת for. הערה: בכל איטרציה המשתנה line בלולאה מצביע על מחרוזת המייצגת שורה מתוך קובץ הטקסט.
- בשורות 3-4 בודקים האם המחרוזת עליה מצביע המשתנה line היא ידידותית או לא. אם כן, השורה מודפסת.
  שימו לב, שלב זה מייצג למעשה **מהלך עבודה עם מחרוזות .**כלומר, אחרי שפותחים קובץ וקוראים את התוכן שלו לתוך משתנה, המשך העבודה עליו מקביל לביצוע פעולות עם מחרוזות.
- בשורה 5, לאחר סיום העבודה עם הקובץ ,**סוגרים את הקובץ.**

פתיחת קובץ לכתיבה

לעיתים נרצה ליצור קובץ חדש או לערוך קובץ קיים, לדוגמה **לכתוב** לתוכו תוכן חדש. מייד נלמד איך לבצע זאת בקלות בפייתון.

למדנו שהפונקציה open() מקבלת פרמטר שמציין את **סוג הגישה לקובץ** (לאחר הפרמטר שמציין את נתיב הגישה לקובץ). הפעם, נעביר לפונקציה כארגומנט את המחרוזת **'w'** שמשמעותה גישה למטרת **כתיבה בלבד.**

שימו לב, פעולת פתיחת קובץ לכתיבה בלבד **דורסת את הקובץ** ,אם הוא קיים. אם שם הקובץ שהעברנו כארגומנט לפונקציה open() אינו קיים , **יווצר קובץ חדש** בשם זה.

דוגמה לפתיחת הקובץ chat.txt לכתיבה:

```
chat_input_file = open("chat.txt", "w")
```

בדוגמה זו פתחנו את הקובץ שנקרא chat.txt בגישת כתיבה בלבד. בסיום הפעולה נקבל בנתיב הקובץ שציינו קובץ ריק שנקרא chat.txt.

כתיבה לקובץ

כדי **לכתוב מידע לתוך קובץ** אפשר להשתמש במתודה **()write** שמאפשרת לכתוב מחרוזות לקבצים.
דוגמה לכתיבת מחרוזת לתוך הקובץ chat.txt:

```
chat_input_file = open("chat.txt", "w")
chat_input_file.write("Me: Why chat, when you can use an owl?")
chat_input_file.close()
```

לאחר שנריץ את הקוד ונפתח את הקובץ chat.txt נבחין שהמחרוזת לעיל מופיעה בו.

זכרו: בסיום העבודה עם קובץ, וללא קשר לסוג הגישה בו בחרתם לפתוח אותו, עליכם לסגור אותו.

להזכירכם, הקובץ chat.txt מכיל את הטקסט הבא:

chat.txt

```
Me: Why chat, when you can use an owl?
```

איזה טקסט יכיל הקובץ chat.txt אם נריץ שוב את קטע הקוד הקודם, בשינוי המחרוזת אותה אנו כותבים לקובץ? חשבו ונסו להריץ בעצמכם.

```
chat_input_file = open("chat.txt", "w")
chat_input_file.write("Harry: Why owl, when you can use a spell?")
chat_input_file.close()
```

אם כך, הקובץ chat.txt מכיל עתה את הטקסט הבא:

chat.txt
```
Me: Why chat, when you can use an owl?
```

פתיחת הקובץ בסוג הגישה "w" דורסת את התוכן הקיים, והמתודה write() כותבת את המחרוזת החדשה לקובץ הריק.

לסיכום נושא העבודה עם קבצים נדגים קריאה מקובץ, ביצוע מניפולציות וכתיבה לקובץ חדש. בדוגמה ניעזר בקובץ המכיל את שמות ספרי הארי פוטר ושנות ההוצאה לאור שלהם, נחלץ מתוכו את שנות ההוצאה לאור ונכתוב אותן לקובץ אחר.

הקובץ הבא נקרא harryPotterBooks.txt. כל שורה בקובץ מייצגת ספר של הארי פוטר והיא מכילה את שם הספר ואת שנת ההוצאה לאור שלו:

harryPotterBooks.txt
```
Harry Potter and the Philosopher's Stone, 2001
Harry Potter and the Chamber of Secrets, 2002
Harry Potter and the Prisoner of Azkaban, 2004
Harry Potter and the Goblet of Fire, 2005
Harry Potter and the Order of the Phoenix, 2007
Harry Potter and the Half-Blood Prince, 2009
Harry Potter and the Deathly Hallows, 2011
```

לפניכם תוכנית שקוראת מן הקובץ harryPotterBooks.txt ויוצרת קובץ חדש שנקרא harryPotterDates.txt אליו נכתבות שנות ההוצאה לאור של ספרי הארי פוטר:

```
COMMA = ", "
dates = ""
books_input_file = open("harryPotterBooks.txt", "r")
for row in books_input_file:
    date = row.split(COMMA)[1]
    dates += date
books_input_file.close()

dates_file = open("harryPotterDates.txt", "w")
dates_file.write(dates)
dates_file.close()
```

פרק 9– קבצים

פירוט השלבים המרכזיים בתוכנית:

- בשורה 2 מגדירים מחרוזת שנקראת dates אליה ישורשרו שנות ההוצאה לאור.
- בשורה 3 הקובץ harryPotterBooks.txt **נפתח לקריאה** בכדי לחלץ ממנו את המידע בנוגע לשנות ההוצאה לאור.
- בשורה 4 **עוברים בלולאה על תוכן הקובץ** .harryPotterBooks.txt בכל איטרציה המשתנה row מצביע על מחרוזת המייצגת שורה מן הקובץ.
- בשורות 5-6 **מחלצים את שנות ההוצאה לאור ומשרשרים אותן** למחרוזת dates. שימו לב, פעולות אלה הן **פעולות על מחרוזות** ,כפי שלמדנו ביחידות קודמות.
  נשים לב ששנות ההוצאה לאור בקובץ ממוקמות בכל שורה לאחר פסיק. לכן, בשורה 5 משתמשים במתודה ()split ומפצלים כל שורה על פי התו פסיק. בעקבות הפעולה מתקבלת רשימה שהאיבר במיקום אחת בה הוא שנת ההוצאה לאור. בשורה 6 התוצאה משורשרת למחרוזת dates.
- בשורות 9-11 **פותחים לכתיבה** את הקובץ .harryPotterDates.txt לאחר מכן **כותבים אל הקובץ** את שנות ההוצאה לאור שקיימות במשתנה dates, ובסיום **סוגרים את הקובץ**.

לאחר ריצת התוכנית יימצא בתיקייה הקובץ harryPotterDates.txt:

harryPotterDates.txt

```
2001
2002
2004
2005
2007
2009
2011
```